Support de Cours — Automatisation de la Virtualisation

Table of Contents

Support de Cours — Automatisation de la Virtualisation

Module : Automatisation et Déploiement d'Infrastructures **Public** : Bac+4 — Spécialité Système, Réseau et Cloud Computing **Volume** : 21 heures — 14 séances de 1h30 **Stack** : Git, Terraform, Ansible, Packer, AWS **Scénario** : Entreprise fictive NovaSphere — startup SaaS en migration IaC

Plan du module

Phase	Séances	Thème
Phase 1 — Fondations	S01-S04	Git, Terraform, AWS EC2, Variables
Phase 2 — Configuration	S05-S07	Ansible, Templates, Rôles, Vault
Phase 3 — Intégration	S09-S11	Automatisation TF→Ansible, Packer AMI, VPC Multi-tier
Phase 4 — Projet final	S12-S14	Projet en binôme + Soutenance

S01 — Culture IaC, Git & Environnement de travail

Contexte NovaSphere

Bienvenue chez **NovaSphere**, une startup SaaS en pleine croissance.

Jusqu'ici, l'équipe infrastructure gérait ses serveurs « à la main » : connexion SSH, installation de paquets un par un, configuration manuelle de chaque machine. Résultat : deux serveurs censés être identiques ne le sont jamais vraiment, personne ne sait exactement ce qui tourne sur quel serveur, et recréer un environnement après un crash prend des jours.

Le CTO a décidé de migrer vers une approche **100 % Infrastructure as Code**, en s'appuyant sur le cloud AWS pour l'hébergement. Vous intégrez l'équipe en tant qu'**ingénieur DevOps junior**. Votre première journée : installer vos outils,

comprendre la philosophie IaC, et prouver que vous pouvez accéder à l'infrastructure cloud.

Objectif du sprint : *Être opérationnel. Zéro excuse pour ne pas committer demain.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Expliquer** les principes fondamentaux de l'Infrastructure as Code (déclaratif vs impératif, idempotence, reproductibilité).
2. **Différencier** les outils de provisioning (Terraform) et de configuration management (Ansible).
3. **Maîtriser** le workflow Git de base : `init`, `add`, `commit`, `push`, `pull`, `.gitignore`.
4. **Valider** l'accès à la console AWS et à l'AWS CLI depuis votre poste.

Partie théorique

1. Le problème : le « serveur flocon de neige »

Imaginez deux serveurs qui devraient être identiques. Un administrateur les a configurés manuellement, à 3 mois d'intervalle. Entre-temps, une version de paquet a changé, un collègue a fait un correctif en urgence sur l'un des deux sans le documenter, et personne ne se souvient exactement des étapes suivies.

Résultat : chaque serveur est **unique**, comme un flocon de neige. C'est le **snowflake server problem**.

Conséquences :

- Impossible de reproduire un environnement à l'identique.
- « Ça marche chez moi, pas en production. »
- Un crash = des jours de reconstruction à tâtons.
- Aucun historique des changements.

2. La solution : Infrastructure as Code (IaC)

L'IaC consiste à **décrire son infrastructure dans des fichiers de code**, versionnés comme du code applicatif.

Trois principes fondamentaux :

Principe	Définition	Exemple concret
Déclaratif	On décrit l'état final souhaité, pas les étapes pour y arriver.	« Je veux 2 instances EC2 avec 4 Go de RAM » — l'outil se débrouille pour y arriver.
Idempotence	Exécuter le même code 1 fois	Si l'instance existe déjà,

Principe	Définition	Exemple concret
	ou 10 fois produit le même résultat.	l'outil ne la recrée pas.
Reproductibilité	Le même code produit la même infrastructure, partout, à chaque fois.	Dev, staging et prod sont identiques car générés par le même code.

Approche déclarative vs impérative :

```
# Impératif (script Bash) — on décrit les ÉTAPES
apt update
apt install -y nginx
systemctl enable nginx
systemctl start nginx

# Déclaratif (Ansible) — on décrit l'ÉTAT SOUHAITÉ
- name: Nginx est installé et actif
  ansible.builtin.package:
    name: nginx
    state: present

- name: Nginx est démarré
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

La différence ? Si Nginx est déjà installé : - Le script Bash **re-exécute tout** (potentiellement avec des effets de bord). - Le code Ansible **constate que c'est déjà bon** et ne fait rien (`ok` au lieu de `changed`).

3. Le paysage des outils

Chez NovaSphere, nous utiliserons principalement 3 outils, avec un quatrième en bonus. Chacun a un rôle précis dans le cycle de vie d'un serveur :

```
┌──────────────
```

```
│              CYCLE DE VIE D'UN SERVEUR                          │
│                                                                 │
│  1. IMAGE         2. PROVISIONING    3. CONFIGURATION          │
│  ┌──────────┐     ┌──────────────┐   ┌──────────────────┐     │
│  │  Packer  │──►│  Terraform   │──►│     Ansible      │     │
│  │          │     │              │   │                  │     │
│  │ Crée une │     │ Crée le      │   │ Configure les    │     │
│  │ image    │     │ serveur      │   │ services         │     │
│  │ réutili- │     │ (EC2, réseau,│   │ (nginx, BDD,     │     │
│  │ sable    │     │  stockage,   │   │  monitoring...)  │     │
│  │ (AMI)    │     │  firewall)   │   │                  │     │
│  └──────────┘     └──────────────┘   └──────────────────┘     │
│                                                                 │
│  Séance 9         Séances 3-4        Séances 5-7               │
└─────────────────────────────────────────────────────────────────┘
```

Outil	Rôle	Catégorie	Quand ?
Terraform	Créer/modifier/détruire des ressources (instances, réseaux, DNS…)	Provisioning	Séances 3-4, 8, 10
Ansible	Configurer les machines (installer des paquets, déployer des apps)	Configuration Management	Séances 5 à 7
Packer	Créer des images machine (AMI) réutilisables	Image Building	Séance 9
Git	Versionner tout le code d'infrastructure	Versioning	Toutes les séances

Règle d'or : Terraform **crée** la machine, Ansible la **configure**. Ne pas mélanger les responsabilités.

4. Git : le dépôt comme source de vérité

En IaC, le dépôt Git **est** l'infrastructure. Si le code dans Git dit « 2 instances EC2 avec Nginx », c'est ce qui doit exister en production. Tout changement passe par un commit.

Pourquoi ? - **Historique** : qui a changé quoi, quand, et pourquoi. - **Rollback** : revenir à un état précédent en une commande. - **Collaboration** : plusieurs personnes travaillent sur la même infra sans se marcher dessus. - **Audit** : on peut prouver l'état de l'infrastructure à un instant T.

```
┌──────────┐    git add     ┌──────────┐   git commit   ┌──────────┐
git push   ┌──────────┐

│ Working  │──────────────►│ Staging  │──────────────►│  Local
│──────────►│  Remote  │

│Directory │               │  Area    │               │  Repo    │
│  Repo    │

│(fichiers)│               │(index)   │               │(.git)    │
│(GitHub)  │

└──────────┘               └──────────┘               └──────────┘
└──────────┘
```

> **Note** : En séance 2, nous approfondirons Git avec les branches, le merge, et le workflow collaboratif. Pour aujourd'hui, on se concentre sur le workflow de base.

TP — Mise en place de l'environnement

Exercice 1 — Installation des outils (15 min)

Installez les outils suivants. Vérifiez chaque installation avec la commande indiquée.

Git :

```
sudo apt update && sudo apt install -y git



# Vérification

git --version

# Attendu : git version 2.x.x
```

Terraform :

```
# Pré-requis

sudo apt-get update && sudo apt-get install -y gnupg software-
properties-common
```

```
# Ajouter la clé GPG HashiCorp
wget -O- https://apt.releases.hashicorp.com/gpg | \
  gpg --dearmor | \
  sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg >
/dev/null


# Ajouter le dépôt
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
  https://apt.releases.hashicorp.com $(grep -oP '(?
<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release -cs) main" | \
  sudo tee /etc/apt/sources.list.d/hashicorp.list


sudo apt update && sudo apt install -y terraform


# Vérification
terraform -version
# Attendu : Terraform v1.x.x
```

Ansible :

```
# Via pipx (méthode recommandée par Ansible sur Ubuntu 24.04+)
sudo apt install -y pipx
pipx install --include-deps ansible
pipx ensurepath


# IMPORTANT : fermez et rouvrez votre terminal, puis vérifiez :
ansible --version
# Attendu : ansible [core 2.x.x]
```

> **Pourquoi `pipx` et pas `pip` ?** Sur Ubuntu 24.04+, Python est « externally managed » : `pip install` échoue ou nécessite un flag `--break-system-packages` qui peut

casser votre système. `pipx` installe Ansible dans un environnement isolé — c'est propre et sans risque.

AWS CLI :

```
# Installation

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

unzip awscliv2.zip

sudo ./aws/install


# Vérification

aws --version

# Attendu : aws-cli/2.x.x
```

Exercice 2 — Création du dépôt Git NovaSphere (25 min)

Vous allez créer le dépôt qui contiendra **toute l'infrastructure** de NovaSphere au fil des séances.

Étape 1 — Configuration initiale de Git :

```
# Configurez votre identité (remplacez par vos informations)

git config --global user.name "___________"

git config --global user.email "___________"


# Branche par défaut

git config --global init.defaultBranch main


# Vérification

git config --list
```

Étape 2 — Création du dépôt local :

```
# Créez le répertoire du projet et initialisez Git

mkdir novasphere-infra
```

```
cd novasphere-infra
git init
```

Étape 3 — Création du fichier `.gitignore` :

Ce fichier est **critique** en IaC. Il empêche de committer des secrets et des fichiers générés.

Créez un fichier `.gitignore` à la racine du dépôt avec le contenu suivant :

```
# Terraform
*.tfstate
*.tfstate.*
.terraform/
*.tfvars
!example.tfvars
crash.log

# Ansible
*.retry

# Secrets et credentials
.env
*.pem
*.key
*_rsa
*_rsa.pub

# AWS
.aws/

# OS
.DS_Store
```

```
Thumbs.db



# IDE

.vscode/

.idea/
```

> **Question** : Pourquoi est-ce que `*.tfstate` et `.env` sont dans le `.gitignore` ? Que se passerait-il si on les committait ?
>
> Répondez dans un commentaire en haut de votre fichier `.gitignore` :

```
# TODO : Expliquez en 2 lignes pourquoi ces fichiers ne doivent JAMAIS être commités

# ___________

# ___________
```

Étape 4 — Création du README :

Créez un fichier `README.md` :

```
# NovaSphere — Infrastructure as Code



## Description

Dépôt d'infrastructure de NovaSphere.

Provisioning via Terraform, configuration via Ansible.



## Stack

- **Provisioning** : Terraform (provider AWS)

- **Configuration** : Ansible

- **Cloud** : AWS (EC2, VPC, Security Groups)

- **Image Building** : Packer (AMI)



## Structure du dépôt

_À compléter au fil des séances._
```

```
## Auteur

- Nom : _____

- Date de création : _____
```

Étape 5 — Premier commit :

```
# Ajoutez tous les fichiers au staging

git add .


# Vérifiez ce qui va être commité

git status


# Committez

git commit -m "feat: initialisation du dépôt novasphere-infra"
```

Étape 6 — Configurer l'authentification vers GitHub/GitLab :

Avant de pouvoir `push`, vous devez prouver votre identité au serveur distant. Deux méthodes existent — choisissez celle que votre professeur recommande.

Méthode A — Clé SSH (recommandée) :

```
# 1. Générer une clé SSH dédiée à Git

ssh-keygen -t ed25519 -C "votre.email@example.com" -f
~/.ssh/id_ed25519_git

# Appuyez sur Entrée pour un passphrase vide (acceptable en contexte
TP)


# 2. Configurer SSH pour utiliser cette clé automatiquement

mkdir -p ~/.ssh

cat >> ~/.ssh/config << 'EOF'

# GitHub

Host github.com

    IdentityFile ~/.ssh/id_ed25519_git

    User git
```

```
# GitLab (décommentez si vous utilisez GitLab)
# Host gitlab.com
#     IdentityFile ~/.ssh/id_ed25519_git
#     User git
EOF

chmod 600 ~/.ssh/config

# 3. Afficher la clé publique — copiez la sortie complète
cat ~/.ssh/id_ed25519_git.pub
```

Dans votre navigateur :

- **GitHub** : Settings → SSH and GPG keys → New SSH key → collez → Add SSH key
- **GitLab** : Preferences → SSH Keys → collez → Add key

```
# 4. Tester la connexion
ssh -T git@github.com       # ou git@gitlab.com
```

Résultat attendu (GitHub) : `Hi <username>! You've successfully authenticated, but GitHub does not provide shell access.`

> **Note** : Le message « does not provide shell access » est normal. SSH sert ici uniquement de canal d'authentification pour Git, pas d'accès terminal. En S04, vous créerez une clé SSH **séparée** pour vous connecter en terminal à vos instances EC2 — on ne mélange pas les usages.

Méthode B — HTTPS + Credential Helper :

Si vous préférez HTTPS (ou si SSH est bloqué par votre réseau) :

```
# 1. Activer le stockage des credentials
git config --global credential.helper store
```

> **Attention** : `credential.helper store` enregistre vos identifiants **en clair** dans `~/.git-credentials`. C'est acceptable pour un TP, mais en entreprise on utilise `credential.helper cache` (stockage temporaire en mémoire, durée limitée) ou un gestionnaire de credentials OS (`osxkeychain` sur macOS, `manager` sur Windows).

```
# Alternative plus sécurisée : stockage en mémoire pendant 4 heures
```

```
git config --global credential.helper 'cache --timeout=14400'
```

Pour l'authentification HTTPS, vous aurez besoin d'un **Personal Access Token** (le mot de passe classique ne fonctionne plus) :

- **GitHub** : Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token → cochez `repo` → Generate → **copiez le token immédiatement** (il ne sera plus visible)
- **GitLab** : Preferences → Access Tokens → Add new token → cochez `write_repository` → Create → copiez le token

Git vous demandera le token lors du premier `push`. Avec `credential.helper store` ou `cache`, il ne le redemandera plus.

Étape 7 — Push vers un dépôt distant :

```
# Créez un dépôt vide sur GitHub ou GitLab (PAS de README, PAS
de .gitignore)

# Puis connectez-le :



# Si Méthode A (SSH) :

git remote add origin git@github.com:<username>/novasphere-infra.git



# Si Méthode B (HTTPS) :

# git remote add origin https://github.com/<username>/novasphere-
infra.git



git branch -M main

git push -u origin main
```

> **Convention de commits NovaSphere** : On utilise des préfixes : - `feat:` → nouvelle fonctionnalité / nouvelle ressource - `fix:` → correction - `docs:` → documentation - `refactor:` → restructuration sans changement fonctionnel

---

Exercice 3 — Configuration AWS CLI (15 min)

AWS est l'environnement cloud principal de NovaSphere. Vous devez configurer l'accès à votre compte AWS depuis votre poste.

Étape 1 — Configuration des credentials :

```
aws configure
# AWS Access Key ID : ___________
# AWS Secret Access Key : ___________
# Default region name : eu-west-3
# Default output format : json
```

Étape 2 — Vérification de votre identité :

```
aws sts get-caller-identity


# Résultat attendu (les valeurs seront différentes) :
# {
#     "UserId": "AIDACKCEVSQ6C2EXAMPLE",
#     "Account": "123456789012",
#     "Arn": "arn:aws:iam::123456789012:user/votre-user"
# }
```

Étape 3 — Exploration des ressources :

```
# Lister les régions disponibles
aws ec2 describe-regions --query "Regions[].RegionName" --output table


# Vérifier qu'aucune instance EC2 ne tourne déjà (éviter les coûts)
aws ec2 describe-instances --query "Reservations[].Instances[].
{ID:InstanceId,State:State.Name,Type:InstanceType}" --output table
```

Étape 4 — Stocker la région dans le projet :

Créez un fichier `.env` à la racine du dépôt pour documenter votre configuration :

```
# Fichier .env — NE JAMAIS COMMITTER
AWS_DEFAULT_REGION="eu-west-3"
AWS_ACCOUNT_ID="___________"
```

> **Vérification** : Exécutez `git status`. Le fichier `.env` doit apparaître comme **non suivi** (ignoré par le `.gitignore`). S'il apparaît en vert (staged), c'est que votre `.gitignore` est mal configuré — corrigez immédiatement.

Note : Les credentials AWS sont stockés dans `~/.aws/credentials`, en dehors du dépôt Git. Ne copiez **jamais** ces credentials dans un fichier du projet.

Variante Proxmox (pour les étudiants disposant d'un environnement Proxmox VE)

Si vous avez accès à un serveur Proxmox VE, vous pouvez également valider cet accès :

1. **Interface web** : Accédez à `https://<IP_PROXMOX>:8006` (acceptez le certificat auto-signé).
2. **SSH** : `ssh admin@<IP_PROXMOX>`, puis `pveversion` pour vérifier la version.
3. **Token API** : Créez un token dans **Datacenter** → **Permissions** → **API Tokens** :
   - User : `admin@pam`, Token ID : `terraform`, décochez « Privilege Separation ».
   - Stockez le token dans votre `.env` :

```
PROXMOX_VE_ENDPOINT="https://<IP>:8006"
PROXMOX_VE_API_TOKEN="admin@pam!terraform=<token-secret>"
```

4. Cet accès sera utile en complément d'AWS pour les séances Terraform (S3-S4).

Exercice 4 — Commit final et validation (5 min)

```
# Vérifiez l'état du dépôt
git status

# Ajoutez les modifications
git add .

# Vérifiez qu'aucun fichier sensible n'est staged
# (.env, .pem, credentials ne doivent PAS apparaître)
git diff --cached --name-only

# Commit final
git commit -m "feat: configuration environnement de travail"

# Push
git push
```

Critères de réussite

#	Critère	Obligatoire ?	Vérifié ?
1	`git --version`, `terraform -version`, `ansible --version`, `aws --version` fonctionnent	Oui	
2	Le dépôt `novasphere-infra` existe sur GitHub/GitLab avec au moins 2 commits	Oui	
3	Le `.gitignore` contient les exclusions Terraform, Ansible, secrets et AWS	Oui	
4	`aws sts get-caller-identity` retourne un résultat valide	Oui	
5	Aucun secret n'est présent dans l'historique Git	Oui	
6	Connexion SSH à Proxmox VE réussie + token API créé	Optionnel	

S02 — Git avancé : Branches, Merge & Workflow collaboratif

Contexte NovaSphere

L'équipe DevOps de NovaSphere s'agrandit. Vous n'êtes plus seul à travailler sur le dépôt d'infrastructure : deux autres ingénieurs viennent d'arriver. Le problème ? Si tout le monde pousse du code directement sur `main`, c'est le chaos garanti : des modifications qui s'écrasent mutuellement, du code cassé qui bloque toute l'équipe, et personne ne sait qui a changé quoi.

Le CTO impose une règle : **plus personne ne travaille directement sur `main`**. Chaque modification passe par une **branche dédiée**, est validée, puis fusionnée. C'est le workflow standard de l'industrie.

Objectif du sprint : *Maîtriser le workflow par branches. À partir d'aujourd'hui, chaque TP = une branche.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Créer, naviguer et supprimer** des branches Git.
2. **Fusionner** une branche dans `main` (fast-forward et merge commit).
3. **Résoudre** un conflit de merge manuellement.
4. **Appliquer** le workflow feature-branch dans le contexte NovaSphere.
5. **Utiliser** les tags pour marquer des versions.

Partie théorique

1. Pourquoi les branches ?

Sans branches, voici ce qui se passe quand 3 personnes travaillent sur `main` :

```
Alice pousse → Bob pull → Bob pousse → CONFLIT avec le push de Clara
                                        Clara pull → Code cassé
                                        Clara doit corriger → Perte de
temps
```

Avec des branches, chacun travaille dans son espace isolé :

```
main          ──────●──────────────────●──────────● (toujours stable)
                    │                  ↑          ↑
feat/ansible ───────┼──●──●──────────merge        │
                    │                             │
feat/terraform ─────┼─────●──●──────────────────merge
                    │
fix/ssh-key ────────┼──●──merge (rapide, une seule correction)
```

Règle d'or : `main` est toujours **fonctionnel**. On ne pousse jamais du code « en cours » sur `main`.

2. Cycle de vie d'une branche

```
┌──────────┐   git switch -c   ┌──────────┐   git add/commit
┌──────────┐

│   main   │──────────────────►│ feat/xxx │──────────────────────►│
feat/xxx │

│          │                   │ (créée)  │                       │
(commits)│

└──────────┘                   └──────────┘
└──────────┘
      │

                                                                  │

                                                              git
switch main

                                                              git
merge feat/xxx

                                                                  │

┌──────────┐   git branch -d   ┌──────────┐                      │
│   main   │◄──────────────────│   main   │◄─────────────────────┘
│(mis à jour)│                  │ (mergé)  │
└──────────┘                   └──────────┘
```

3. Deux types de merge

Fast-forward (avance rapide) : quand `main` n'a pas bougé depuis la création de la branche. Git déplace simplement le pointeur.

```
Avant :  main ──●──          feat ──●──●──●

Après :  main ──●──●──●──●   (main avance jusqu'à feat)
```

Merge commit (fusion à 3 voies) : quand `main` a évolué en parallèle. Git crée un commit de fusion qui combine les deux historiques.

```
Avant :  main ──●──●──●         feat ──●──●

Après :  main ──●──●──●──M     (M = commit de merge qui combine les
deux)

                          ↑

                     fusion des deux branches
```

4. Les conflits

Un conflit survient quand **deux branches modifient la même ligne du même fichier**. Git ne peut pas deviner quelle version garder — c'est à vous de trancher.

Un fichier en conflit ressemble à ça :

```
<<<<<<< HEAD

version_de_main

=======

version_de_la_branche

>>>>>>> feat/ma-branche
```

Résolution : vous supprimez les marqueurs (<<<<, ====, >>>>), gardez le bon contenu, puis `git add` + `git commit`.

5. Convention NovaSphere pour les branches

Préfixe	Usage	Exemple
`feat/`	Nouvelle fonctionnalité ou ressource	`feat/ec2-web-server`
`fix/`	Correction d'un bug	`fix/security-group-port`
`docs/`	Documentation uniquement	`docs/readme-architecture`
`refactor/`	Restructuration sans changement fonctionnel	`refactor/variables-extraction`

> **Important** : chaque TP des séances suivantes sera fait dans une branche dédiée (ex : `feat/s03-terraform-ec2`), puis mergé dans `main` à la fin.

TP — Maîtriser les branches

Vous travaillez dans le dépôt `novasphere-infra` créé en S1. Assurez-vous d'être à jour :

```
cd ~/novasphere-infra

git pull

git status   # Doit afficher "nothing to commit, working tree clean"
```

Exercice 1 — Créer et naviguer entre les branches (15 min)

Étape 1 — Lister les branches existantes :

```
git branch
```

```
# Attendu : * main (l'étoile indique la branche courante)
```

Étape 2 — Créer une branche et basculer dessus :

```
# Créer et basculer en une commande
git switch -c feat/documentation

# Vérifier
git branch
# Attendu :
# * feat/documentation
#   main
```

Étape 3 — Faire des modifications sur la branche :

Créez un fichier `docs/architecture.md` :

```
mkdir -p docs
# Architecture NovaSphere

## Environnement
- **Cloud** : AWS (région eu-west-3)
- **Provisioning** : Terraform
- **Configuration** : Ansible
- **Image Building** : Packer

## Schéma cible
_À compléter au fil des séances._
git add docs/
git commit -m "docs: ajout du fichier architecture"
```

Étape 4 — Ajouter un deuxième commit sur la branche :

Modifiez le `README.md` pour ajouter un lien vers la doc :

```
## Structure du dépôt
```

```
- `docs/` — Documentation d'architecture
```

TODO : Remplacez la ligne `_À compléter au fil des séances._` du README par le texte ci-dessus, puis committez :

```
git add README.md

git commit -m "____________"
```

Quel préfixe de commit utilisez-vous ici ? Pourquoi ?

Étape 5 — Visualiser l'historique :

```
git log --oneline --graph --all

# Vous devriez voir feat/documentation en avance sur main
```

Exercice 2 — Merger une branche (fast-forward) (10 min)

Étape 1 — Revenir sur `main` :

```
git switch main



# Vérifiez : le dossier docs/ a disparu ! (il n'existe que sur la
branche)

ls docs/

# Erreur : No such file or directory
```

Étape 2 — Fusionner :

```
git merge feat/documentation

# Attendu : "Fast-forward" — main n'a pas bougé, donc Git avance
simplement le pointeur
```

Étape 3 — Vérifier :

```
ls docs/

# Le dossier est revenu !



git log --oneline --graph --all

# feat/documentation et main pointent maintenant vers le même commit
```

Étape 4 — Supprimer la branche mergée :

```
git branch -d feat/documentation
```

```
# Attendu : Deleted branch feat/documentation


git branch
# Attendu : * main (seule branche restante)
```

Étape 5 — Pusher :

```
git push
```

Exercice 3 — Provoquer et résoudre un conflit (20 min)

C'est l'exercice le plus important de la séance. Vous allez **volontairement** créer un conflit pour apprendre à le résoudre.

Étape 1 — Créer une branche :

```
git switch -c feat/stack-update
```

Étape 2 — Modifier le README sur la branche :

Dans `README.md`, changez la section Stack :

```
## Stack
- **Provisioning** : Terraform 1.14+ (provider AWS)
- **Configuration** : Ansible 2.17+ (via pipx)
- **Cloud** : AWS (EC2, VPC, Security Groups, IAM)
- **Image Building** : Packer 1.11+ (AMI)
git add README.md
git commit -m "docs: mise à jour des versions dans la stack"
```

Étape 3 — Revenir sur `main` et y faire une modification concurrente :

```
git switch main
```

Modifiez la **même section** Stack du README **différemment** :

```
## Stack
- **Provisioning** : Terraform (provider AWS + Proxmox optionnel)
- **Configuration** : Ansible
- **Cloud** : AWS (EC2, VPC, Security Groups)
- **Image Building** : Packer (AMI)
```

```
- **CI/CD** : GitHub Actions (à venir)

git add README.md

git commit -m "docs: ajout CI/CD et Proxmox dans la stack"
```

Étape 4 — Tenter le merge :

```
git merge feat/stack-update

# Attendu : CONFLICT (content): Merge conflict in README.md

# Automatic merge failed; fix conflicts and then commit the result.
```

Étape 5 — Observer le conflit :

```
git status

# Affiche : "both modified: README.md"



# Ouvrez README.md dans votre éditeur — vous verrez les marqueurs de conflit

cat README.md
```

Étape 6 — Résoudre le conflit :

> **TODO** : Ouvrez `README.md` et combinez les deux versions de manière logique. Vous devez : - Garder les numéros de version de la branche `feat/stack-update` - Garder la ligne CI/CD de `main` - Garder la mention Proxmox optionnel - Supprimer tous les marqueurs de conflit (<<<<, ====, >>>>)
>
> Résultat attendu :

```
## Stack

___________

___________

___________

___________

___________
```

Étape 7 — Finaliser le merge :

```
git add README.md

git commit -m "merge: combinaison stack versions + CI/CD"
```

```
# Vérifier l'historique
git log --oneline --graph --all
# Vous devriez voir le commit de merge avec deux parents
```

Étape 8 — Nettoyer et pusher :

```
git branch -d feat/stack-update
git push
```

Exercice 4 — Workflow feature-branch complet (15 min)

Vous allez simuler le workflow que vous utiliserez à chaque TP à partir de la S3.

Étape 1 — Créer la branche pour un nouveau « TP » :

```
git switch -c feat/s02-gitignore-avance
```

Étape 2 — Améliorer le `.gitignore` :

Ajoutez ces lignes à votre `.gitignore` existant :

```
# Packer
packer_cache/
*.pkr.hcl.lock

# Logs
*.log

# Fichiers temporaires
*.tmp
*.bak
*.swp
*~
git add .gitignore
git commit -m "feat: enrichissement du .gitignore pour Packer et fichiers temporaires"
```

Étape 3 — Créer un tag de version :

```
# D'abord, merger dans main
git switch main
git merge feat/s02-gitignore-avance
git branch -d feat/s02-gitignore-avance

# Créer un tag
git tag -a v0.2.0 -m "S02: git avancé terminé, .gitignore enrichi"

# Vérifier
git tag
# Attendu : v0.2.0
```

Étape 4 — Pusher le tout (commits + tags) :

```
git push
git push --tags
```

> **Convention de tags NovaSphere** : - `v0.X.0` → fin de séance X (ex : `v0.1.0` = S1, `v0.2.0` = S2) - Permet de revenir à l'état exact du dépôt à la fin de chaque séance

---

Exercice 5 — Commit final et validation (5 min)

```
# État final attendu
git log --oneline --graph --all

# Vérification du tag
git show v0.2.0

# Vérification des branches (aucune branche fantôme)
git branch
# Attendu : * main (seule branche)
```

> **TODO** : Revenez en S1 en une commande. Qu'observez-vous ? Comment revenez-vous à la version actuelle ?

```
git checkout v0.1.0
```

```
# Que dit Git ? ___________

# Comment revenir ? ___________
```

Critères de réussite

#	Critère	Vérifié ?
1	Au moins 3 branches ont été créées et mergées pendant le TP	
2	Un conflit a été résolu correctement (pas de marqueurs <<<< résiduels)	
3	L'historique montre un merge commit (pas uniquement des fast-forward)	
4	Le tag `v0.2.0` existe et est pushé sur le dépôt distant	
5	Aucune branche orpheline ne traîne (`git branch` = `* main` uniquement)	
6	Le `.gitignore` contient les exclusions Packer et fichiers temporaires	

S03 — Terraform : Premiers pas (Local & AWS)

Contexte NovaSphere

L'architecte infra de NovaSphere vous convoque :

« Avant de toucher à quoi que ce soit en production, vous allez apprendre Terraform. Pas question de casser l'infra parce que vous ne comprenez pas ce que fait `apply`. On commence en local, sur vos postes — zéro risque. Ensuite, quand vous maîtriserez le cycle de base, on lance une VM sur AWS pour de vrai. »

Objectif du sprint : *Comprendre le moteur Terraform, puis l'utiliser pour créer une vraie instance EC2 sur AWS.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Comprendre** la syntaxe HCL et les blocs fondamentaux (`terraform`, `provider`, `resource`).
2. **Exécuter** le cycle de vie complet : `terraform init` → `plan` → `apply` → `destroy`.
3. **Lire** un plan Terraform et identifier les actions prévues (`+`, `-`, `~`).
4. **Expliquer** le rôle du fichier `terraform.tfstate`.
5. **Déployer** une instance EC2 sur AWS via Terraform.

Partie théorique

1. Terraform : un outil déclaratif

Terraform ne décrit pas **comment** créer une infrastructure (étape par étape), mais **quel état final** on veut atteindre. C'est le principe **déclaratif**.

```
┌──────────────────────────────────────────────────────────┐
│                    APPROCHE DÉCLARATIVE                  │
│                                                          │
│   Vous écrivez :          Terraform calcule :            │
│   "Je veux une VM         "Il faut créer une VM          │
│    avec 4 Go de RAM"       car elle n'existe pas encore" │
│                                                          │
│   Vous ré-exécutez :      Terraform constate :           │
│   "Je veux une VM         "La VM existe déjà avec        │
│    avec 4 Go de RAM"       4 Go → rien à faire"          │
│                                                          │
│   Vous modifiez :         Terraform calcule :            │
│   "Je veux une VM         "Il faut modifier la RAM       │
│    avec 8 Go de RAM"       de 4 Go → 8 Go"              │
└──────────────────────────────────────────────────────────┘
```

2. Le langage HCL — Les blocs fondamentaux

Terraform utilise le langage **HCL** (HashiCorp Configuration Language). Tout fichier `.tf` est composé de **blocs** :

```
# Bloc 1 : TERRAFORM — Configuration globale
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

# Bloc 2 : PROVIDER — Connexion à une API
provider "local" {}

# Bloc 3 : RESOURCE — Un objet à créer/gérer
resource "local_file" "hello" {
  content  = "Bienvenue chez NovaSphere !"
  filename = "${path.module}/hello.txt"
}
```

Bloc	Rôle	Analogie
`terraform {}`	Configuration de Terraform lui-même (versions, providers)	Le « cahier des charges »
`provider "<nom>" {}`	Connexion à un service (AWS, local, Proxmox…)	Le « connecteur » vers une API
`resource "<type>" "<nom>" {}`	Un objet concret à créer et gérer	L'« objet » à fabriquer

Points de syntaxe : - Les chaînes sont entre guillemets doubles `"..."`. - Les commentaires commencent par #. - L'indentation est par convention de 2 espaces. - `${path.module}` est une référence built-in qui donne le chemin du répertoire courant.

3. Le cycle de vie Terraform

```
  |   init    |──────►|   plan    |──────►|  apply    |──────►| destroy  |
  └───────────┘       └───────────┘       └───────────┘       └──────────┘

      |                 |                 |                 |

  Télécharge        Compare le        Exécute le        Supprime toutes
  les providers     code (.tf) au     plan : crée,      les ressources
  et initialise     state actuel      modifie ou        gérées
  le projet         → affiche         détruit les
                    un plan           ressources
```

Commande	Action	Quand l'utiliser
`terraform init`	Télécharge les providers, initialise le backend	Au début, puis après chaque ajout de provider
`terraform plan`	Calcule les différences, affiche ce qui va changer	**Toujours** avant un `apply`
`terraform apply`	Exécute les changements	Quand le plan vous convient
`terraform destroy`	Supprime **toutes** les ressources gérées	Pour nettoyer un environnement

4. Le State — La mémoire de Terraform

Terraform stocke l'état de votre infrastructure dans `terraform.tfstate` (JSON).

Pourquoi ? Sans le state, Terraform ne sait pas ce qui existe déjà → il essaie de tout recréer → erreur.

Analogie : Le code `.tf` dit « je veux 3 chaises ». Le state dit « il y en a 2 ». Terraform calcule : « il faut en ajouter 1 ».

```
  Code (.tf)                 State (.tfstate)           Infrastructure
réelle

  ┌──────────┐            ┌──────────────┐
┌──────────────┐

  | Ce que je  |            | Ce que TF      |           | Ce qui existe
|

  | VEUX       |◄── diff ──►| CROIT exister  |── sync ──►| VRAIMENT
|

  └──────────┘            └──────────────┘
└──────────────┘
```

> **Règle d'or** : `terraform.tfstate` ne doit **JAMAIS** être commité dans Git. Il est déjà dans votre `.gitignore` (vérifié en S1).

TP — Partie 1 : Terraform en local (20 min)

Workflow Git : Créez une branche dédiée avant de commencer.

```
cd ~/novasphere-infra
git switch -c feat/s03-terraform
```

Exercice 1 — Premier cycle complet

Étape 1 — Créez un répertoire pour cette séance :

```
mkdir -p terraform/lab-local
cd terraform/lab-local
```

Étape 2 — Créez un fichier `main.tf` :

```
# main.tf — S03 : Premier déploiement Terraform


terraform {
  required_providers {
    local = {
      source  = "___________"          # TODO 1 : Chemin du provider
local HashiCorp ?
      version = "~> 2.5"
    }
  }
}


provider "local" {}


resource "local_file" "novasphere_welcome" {
  content  = "___________"              # TODO 2 : Écrivez un message
de bienvenue NovaSphere
  filename = "${path.module}/welcome.txt"
}
```

Indice : Les providers officiels suivent le format `hashicorp/<nom>`.

Étape 3 — Exécutez le cycle complet :

```
terraform init          # Télécharge le provider
terraform plan          # Visualisez le plan
terraform apply         # Créez le fichier (tapez "yes")
cat welcome.txt         # Vérifiez le contenu
```

Étape 4 — Testez l'idempotence :

```
terraform plan
```

Question : Que dit le plan cette fois-ci ? Pourquoi ?

Réponse : _____

Étape 5 — Modifiez et observez la diff :

Changez le contenu de la resource :

```
  content  = "NovaSphere — Infrastructure as Code v2.0"
terraform plan
```

Question : Quels symboles voyez-vous dans le plan ? Que signifient-ils ?

- `+` signifie : _____
- `-` signifie : _____
- `~` signifie : _____
- `-/+` signifie : _____

```
terraform apply -auto-approve    # Appliquez sans confirmation (TP uniquement !)
```

Étape 6 — Détruisez et nettoyez :

```
terraform destroy                # Confirmez avec "yes"
ls welcome.txt                   # Doit échouer : fichier supprimé
```

Checkpoint : Vous maîtrisez le cycle `init → plan → apply → destroy`. Les mêmes commandes vont maintenant créer une vraie VM sur AWS.

TP — Partie 2 : Première instance EC2 sur AWS (40 min)

Le changement : juste le provider

```
  Partie 1 (Local)                    Partie 2 (AWS)

  ┌──────────────────────┐            ┌──────────────────────┐
```

```
  | provider "local"  |                | provider "aws"    |
  |                   |                |   region = "..."  |
  | resource          |                |                   |
  |   "local_file"    |                | resource          |
  |   → fichier texte |                |   "aws_instance"  |
  |                   |                |   → serveur EC2   |
  | terraform apply   |                | terraform apply   |
  | terraform destroy |                | terraform destroy |
  └───────────────────┘                └───────────────────┘


  Même workflow. Même commandes. Résultat différent.
```

Exercice 2 — Déployer une EC2

Étape 1 — Créez un nouveau répertoire :

```
cd ~/novasphere-infra/terraform
mkdir -p lab-aws
cd lab-aws
```

Étape 2 — Vérifiez que votre AWS CLI est bien configuré :

```
aws sts get-caller-identity
# Doit afficher votre Account ID et votre ARN
# Si erreur → vérifiez votre configuration S1 (aws configure)
```

Étape 3 — Créez le fichier `main.tf` :

```
# main.tf — S03 : Première instance EC2


terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
```

```
    }
  }
}

provider "aws" {
  region = "eu-west-3"    # Paris
}

# --- Recherche de l'AMI Ubuntu 24.04 la plus récente ---
# Les AMI (Amazon Machine Images) changent régulièrement.
# Plutôt que de noter un ID qui sera périmé dans 2 semaines,
# on demande à AWS de nous donner le plus récent.
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]    # Canonical (éditeur d'Ubuntu)

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-
server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# --- Security Group : autoriser SSH ---
```

```
resource "aws_security_group" "novasphere_ssh" {
  name        = "___________"        # TODO 3 : Donnez un nom
descriptif
  description = "Autoriser SSH depuis n'importe où"

  ingress {
    description = "SSH"
    from_port   = ___________        # TODO 4 : Quel port pour SSH ?
    to_port     = ___________        # TODO 4 : Même port
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"               # -1 = tous les protocoles
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name    = "novasphere-ssh"
    Project = "NovaSphere"
  }
}

# --- Instance EC2 ---
resource "aws_instance" "web" {
  ami                     = data.aws_ami.ubuntu.id
```

```
  instance_type          = "____________"       # TODO 5 : Quel type
d'instance économique ?

  vpc_security_group_ids = [aws_security_group.novasphere_ssh.id]


  tags = {

    Name    = "novasphere-web"

    Project = "NovaSphere"

    Session = "S03"

  }

}


# --- Outputs : afficher les infos utiles après apply ---

output "instance_id" {

  description = "ID de l'instance EC2"

  value       = aws_instance.web.id

}


output "public_ip" {

  description = "Adresse IP publique"

  value       = aws_instance.web.____________   # TODO 6 : Quel
attribut pour l'IP publique ?

}


output "ami_used" {

  description = "AMI utilisée"

  value       = data.aws_ami.ubuntu.id

}
```

Indices : - TODO 3 : Un nom qui décrit la fonction du Security Group (convention : `novasphere-xxx`). - TODO 4 : Le port standard SSH est bien connu. - TODO 5 :

L'instance la plus petite en génération 3 (format `tX.micro`). - TODO 6 : L'attribut s'appelle `public_ip`.

Étape 4 — Déployez :

```
terraform init           # Télécharge le provider AWS (plus gros que le local !)

terraform plan           # Lisez attentivement — combien de ressources seront créées ?
```

Question : Le plan annonce la création de combien de ressources ? Lesquelles ?

Réponse : _____

```
terraform apply          # Tapez "yes" — l'instance est créée !
```

Note : La création d'une EC2 prend 30-60 secondes. C'est normal.

Étape 5 — Vérifiez dans la console AWS :

Ouvrez https://eu-west-3.console.aws.amazon.com/ec2/ et vérifiez que votre instance `novasphere-web` apparaît avec le statut **Running**.

Étape 6 — Vérifiez via le CLI :

```
# L'IP publique est affichée dans les outputs de terraform apply

# Vous pouvez aussi la récupérer avec :

terraform output public_ip
```

Étape 7 — Observez le state :

```
# Taille du state (il contient maintenant des infos AWS)

ls -lh terraform.tfstate



# Nombre de ressources gérées

terraform state list

# Attendu :

#   data.aws_ami.ubuntu

#   aws_instance.web

#   aws_security_group.novasphere_ssh
```

Exercice 3 — Modifier et détruire (10 min)

Étape 1 — Ajoutez un tag à l'instance :

```
  tags = {
    Name        = "novasphere-web"
    Project     = "NovaSphere"
    Session     = "S03"
    Environment = "dev"              # Nouveau tag
  }
terraform plan
```

> **Question** : Le plan dit `~ update in-place`. Que signifie ~ ici ? Pourquoi Terraform ne recrée-t-il pas l'instance ?
>
> Réponse : _____

```
terraform apply -auto-approve
```

Étape 2 — Détruisez tout :

```
terraform destroy
# Tapez "yes"
```

> **Important** : `terraform destroy` en fin de séance est **obligatoire** pour les ressources AWS. Chaque minute d'instance active consomme vos crédits. Vérifiez dans la console AWS que l'instance est bien **terminated**.

> **Encadré optionnel — Provider Proxmox**
>
> Si vous disposez d'un Proxmox VE, vous pouvez réaliser le même exercice avec le provider `bpg/proxmox`. Le workflow est identique :

```
terraform {
  required_providers {
    proxmox = {
      source  = "bpg/proxmox"
      version = "~> 0.74"
    }
  }
}
```

```
provider "proxmox" {
  endpoint = "https://VOTRE_IP:8006"
  api_token = "votre-token"
  insecure = true
}
```

C'est exactement le même principe : on change le provider et le type de resource, mais `init`, `plan`, `apply`, `destroy` restent identiques. C'est la puissance de Terraform : **un seul outil, n'importe quel fournisseur**.

Exercice 4 — Commit et tag (5 min)

```
cd ~/novasphere-infra

# Vérifiez que les .tfstate ne seront pas commités
grep "tfstate" .gitignore

# Ajoutez et committez
git add terraform/
git commit -m "feat: terraform premiers pas (local + AWS EC2)"

# Mergez dans main et taguez
git switch main
git merge feat/s03-terraform
git branch -d feat/s03-terraform
git tag -a v0.3.0 -m "S03: terraform premiers pas terminé"
git push && git push --tags
```

Critères de réussite

#	Critère	Vérifié ?
1	Le cycle complet local	

#	Critère	Vérifié ?
	fonctionne (init/plan/apply/destroy)	
2	L'instance EC2 a été créée et visible dans la console AWS	
3	Les outputs affichent l'ID de l'instance et l'IP publique	
4	`terraform plan` sans modification affiche « No changes »	
5	L'instance EC2 a été **détruite** en fin de séance	
6	Le code est commité, branche mergée, tag `v0.3.0` pushé	
7	Aucun `terraform.tfstate` dans le dépôt Git	

S04 — Terraform avancé : Variables, Outputs & Structure

Contexte NovaSphere

Le lead tech de NovaSphere a revu votre code Terraform de la S3 :

« Ça marche, c'est bien. Mais le type d'instance, la région, les tags — tout est écrit en dur. Si demain on veut déployer la même infra en staging avec une instance plus grosse, il faut modifier le code à 10 endroits. C'est ingérable. Il faut paramétrer tout ça. Et tant qu'on y est, structurez le projet correctement — un main.tf de 80 lignes, c'est illisible. »

Objectif du sprint : *Zéro valeur en dur. Le même code, des déploiements différents.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Déclarer** et utiliser des variables avec typage et validation.
2. **Structurer** un projet Terraform en fichiers séparés (`providers.tf`, `variables.tf`, `main.tf`, `outputs.tf`).
3. **Utiliser** des fichiers `.tfvars` pour déployer différents environnements.
4. **Créer** une key pair SSH via Terraform pour préparer la connexion Ansible.

Partie théorique

1. Les variables — Paramétrer le code

Les variables permettent de **réutiliser le même code** avec des valeurs différentes. C'est comme les paramètres d'une fonction.

```
# Déclaration (dans variables.tf)
variable "instance_type" {
  description = "Type d'instance EC2"
  type        = string
  default     = "t3.micro"
}

# Utilisation (dans main.tf)
resource "aws_instance" "web" {
  instance_type = var.instance_type    # ← Référence à la variable
}
```

Les types disponibles :

Type	Exemple	Usage
`string`	`"t3.micro"`	Texte
`number`	`22`	Nombres (ports, tailles…)
`bool`	`true`	Activer/désactiver un comportement
`list(string)`	`["web", "dev"]`	Liste de valeurs
`map(string)`	`{ env = "dev" }`	Paires clé-valeur

Comment passer les valeurs (ordre de priorité croissant) :

```
1. default dans la déclaration        ← priorité la plus faible
```

```
2. Fichier terraform.tfvars            (chargé automatiquement)

3. Fichier nommé (-var-file="prod.tfvars")

4. Variable d'environnement (TF_VAR_instance_type=...)

5. Ligne de commande (-var="instance_type=...")   ← priorité la plus forte
```

Validation :

```
variable "instance_type" {

  description = "Type d'instance EC2"

  type        = string

  default     = "t3.micro"



  validation {

    condition     = contains(["t3.micro", "t3.small", "t3.medium"], var.instance_type)

    error_message = "Le type d'instance doit être t3.micro, t3.small ou t3.medium."

  }

}
```

2. Les outputs — Exposer l'information

Les outputs affichent des informations **après un `apply`** : IP publique, ID de l'instance, etc.

```
# Dans outputs.tf

output "public_ip" {

  description = "IP publique de l'instance"

  value       = aws_instance.web.public_ip

}
```

Après `terraform apply` :

```
Outputs:



public_ip = "13.38.xx.xx"
```

Les outputs servent aussi à **transmettre des informations entre modules** (séance 8+).

3. Structure d'un projet Terraform

```
terraform/aws/
├── providers.tf         # Provider AWS + versions
├── main.tf              # Ressources (EC2, SG, Key Pair)
├── variables.tf         # Déclaration de TOUTES les variables
├── outputs.tf           # Déclaration de TOUS les outputs
├── terraform.tfvars     # Valeurs par défaut (gitignored !)
├── prod.tfvars          # Valeurs de prod (gitignored !)
├── terraform.tfvars.example  # Exemple SANS secrets (commité !)
└── .terraform.lock.hcl  # Lock file des versions (commité)
```

> **Convention** : Terraform charge automatiquement **tous les fichiers `.tf`** d'un répertoire. L'ordre n'a pas d'importance — Terraform construit un graphe de dépendances. La séparation en fichiers est une **convention de lisibilité**, pas une obligation technique.

TP — Refactoring du code AWS

> **Workflow Git** :

```
cd ~/novasphere-infra
git switch -c feat/s04-terraform-variables
```

Exercice 1 — Éclater le code en fichiers (15 min)

Objectif : Transformer le `main.tf` monolithique de la S3 en structure multi-fichiers.

Étape 1 — Créez un nouveau répertoire pour le code refactoré :

```
mkdir -p terraform/aws
cd terraform/aws
```

> Le répertoire `terraform/lab-aws/` de S3 reste tel quel (c'est votre historique). On repart proprement dans `terraform/aws/`.

Étape 2 — Créez `providers.tf` :

```
# providers.tf — Configuration du provider AWS


terraform {
  required_providers {
```

```
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Étape 3 — Créez `variables.tf` :

```
# variables.tf — Variables du projet NovaSphere

variable "aws_region" {
  description = "Région AWS"
  type        = string
  default     = "eu-west-3"
}

variable "project_name" {
  description = "Nom du projet (utilisé dans les tags et les noms)"
  type        = string
  default     = "novasphere"

  validation {
    condition     = can(regex("^[a-z][a-z0-9-]+$", var.project_name))
    error_message = "Le nom du projet doit être en minuscules,
lettres, chiffres et tirets uniquement."
  }
```

```
}


variable "environment" {
  description = "Environnement de déploiement"
  type        = string
  default     = "dev"

  validation {
    condition     = ___________    # TODO 1 : Limiter à "dev",
"staging" ou "prod"
    error_message = "L'environnement doit être dev, staging ou prod."
  }
}


variable "instance_type" {
  description = "Type d'instance EC2"
  type        = ___________        # TODO 2 : Quel type ?
  default     = "t3.micro"

  validation {
    condition     = contains(["t3.micro", "t3.small", "t3.medium"],
var.instance_type)
    error_message = "Le type d'instance doit être t3.micro, t3.small
ou t3.medium."
  }
}


variable "ssh_public_key_path" {
  description = "Chemin vers la clé publique SSH"
```

```
  type        = string
  default     = "~/.ssh/id_rsa.pub"
}

variable "allowed_ssh_cidr" {
  description = "Bloc CIDR autorisé pour SSH"
  type        = string
  default     = "0.0.0.0/0"      # Tout le monde (TP uniquement !)
}
```

Étape 4 — Créez `main.tf` :

```
# main.tf — Ressources NovaSphere (paramétré)

# --- Nommage local ---
locals {
  name_prefix = "${var.project_name}-${var.environment}"
  common_tags = {
    Project     = var.project_name
    Environment = var.environment
    ManagedBy   = "terraform"
  }
}

# --- AMI Ubuntu 24.04 ---
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
```

```
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-
server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# --- Key Pair SSH ---
resource "aws_key_pair" "deployer" {
  key_name   = "${local.name_prefix}-key"
  public_key = file(var.ssh_public_key_path)

  tags = local.common_tags
}

# --- Security Group ---
resource "aws_security_group" "web" {
  name        = "${local.name_prefix}-sg"
  description = "Security Group pour ${local.name_prefix}"

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
```

```
    cidr_blocks = [____________]    # TODO 3 : Quelle variable pour le
CIDR SSH ?
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-sg"
  })
}

# --- Instance EC2 ---
resource "aws_instance" "web" {
  ami                    = ____________                    # TODO 4 :
Référence à la data source AMI
  instance_type          = var.instance_type
  key_name               = aws_key_pair.deployer.key_name
  vpc_security_group_ids = [aws_security_group.web.id]

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-web"
  })
}
```

Étape 5 — Créez `outputs.tf` :

```
# outputs.tf — Informations exposées après déploiement


output "instance_id" {
  description = "ID de l'instance EC2"
  value       = aws_instance.web.id
}


output "public_ip" {
  description = "___________"        # TODO 5 : Décrivez cet output
  value       = aws_instance.web.public_ip
}


output "ssh_command" {
  description = "Commande SSH pour se connecter"
  value       = "ssh -i ${var.ssh_public_key_path} ubuntu@$
{aws_instance.web.public_ip}"
}


output "ami_id" {
  description = "AMI utilisée"
  value       = data.aws_ami.ubuntu.id
}


output "security_group_id" {
  description = "ID du Security Group"
  value       = aws_security_group.web.id
}
```

Exercice 2 — Générer la clé SSH et déployer (15 min)

Étape 1 — Générez une clé SSH (si vous n'en avez pas encore) :

```
# Vérifiez d'abord si vous en avez une
ls ~/.ssh/id_rsa.pub

# Si le fichier n'existe pas, générez une clé :
ssh-keygen -t rsa -b 4096 -C "novasphere-deploy" -f ~/.ssh/id_rsa -N
""
```

Étape 2 — Créez le fichier `terraform.tfvars` :

```
# terraform.tfvars — Valeurs pour l'environnement dev

project_name    = "novasphere"
environment     = "dev"
instance_type   = "t3.micro"
aws_region      = "eu-west-3"
```

Étape 3 — Créez aussi un fichier d'exemple (celui-ci sera commité) :

```
# terraform.tfvars.example — Exemple de configuration (SANS secrets)

project_name    = "novasphere"
environment     = "dev"
instance_type   = "t3.micro"
aws_region      = "eu-west-3"
# ssh_public_key_path = "~/.ssh/id_rsa.pub"
# allowed_ssh_cidr    = "203.0.113.0/24"
```

Étape 4 — Déployez :

```
terraform init
terraform plan
```

> **Question** : Combien de ressources le plan prévoit-il de créer ? Lesquelles ?
>
> Réponse : _____

```
terraform apply
```

Étape 5 — Testez la connexion SSH :

```
# Récupérez la commande SSH depuis les outputs
terraform output ssh_command


# Testez (attendez ~30 secondes que l'instance boote)
ssh -i ~/.ssh/id_rsa ubuntu@$(terraform output -raw public_ip)
```

> **Question** : Si la connexion SSH fonctionne, quel a été le rôle de la resource `aws_key_pair` ?
>
> Réponse : _____

```
# Déconnectez-vous de l'instance
exit
```

Exercice 3 — Le pouvoir des tfvars (10 min)

Objectif : Prouver que le même code peut déployer des environnements différents.

Étape 1 — Créez un fichier `staging.tfvars` :

```
# staging.tfvars — Valeurs pour simuler un environnement staging



project_name  = "novasphere"
environment   = "staging"
instance_type = "___________"     # TODO 6 : Choisissez un type plus
gros que dev
```

> **Rappel** : Un seul state par répertoire. Pour déployer un second environnement, il faut d'abord détruire le premier (ou utiliser des workspaces/modules — séance 8+).

Étape 2 — Détruisez le dev :

```
terraform destroy -auto-approve
```

Étape 3 — Déployez le staging :

```
terraform apply -var-file="staging.tfvars"
```

Étape 4 — Observez les différences :

```
terraform output
```

```
# Le nom contient "staging" au lieu de "dev"
# Le type d'instance est celui que vous avez choisi
```

Étape 5 — Testez la validation :

```
# Essayez un type d'instance non autorisé
terraform plan -var="instance_type=t3.xlarge"
```

> **Question** : Quel message obtenez-vous ? Quelle partie du code provoque cette erreur ?
>
> Réponse : _____

Étape 6 — Nettoyez :

```
terraform destroy -auto-approve
```

Exercice 4 — Exploration du state (5 min)

Avant de détruire (ou après un apply), explorez :

```
# Lister toutes les ressources gérées
terraform state list

# Voir le détail d'une ressource
terraform state show aws_instance.web

# Tous les outputs en JSON
terraform output -json
```

> **Question** : `terraform state show aws_instance.web` affiche beaucoup plus d'attributs que votre code `.tf`. D'où viennent ces attributs supplémentaires ?
>
> Réponse : _____

Exercice 5 — Commit final (5 min)

```
cd ~/novasphere-infra

# Vérifiez que les tfvars et tfstate ne sont PAS dans le staging
git status
```

```
# Commit
git add terraform/aws/
git add terraform/aws/terraform.tfvars.example   # Celui-ci OUI
git commit -m "refactor: terraform variables, structure multi-fichiers, key pair SSH"


# Merge et tag
git switch main
git merge feat/s04-terraform-variables
git branch -d feat/s04-terraform-variables
git tag -a v0.4.0 -m "S04: terraform variables et structure"
git push && git push --tags
```

> **Question** : Pourquoi commite-t-on `terraform.tfvars.example` mais PAS `terraform.tfvars` ?
>
> Réponse : _____

Critères de réussite

#	Critère	Vérifié ?
1	Le projet est structuré en 4 fichiers : `providers.tf`, `main.tf`, `variables.tf`, `outputs.tf`	
2	Aucune valeur en dur dans `main.tf` (tout paramétré via variables)	
3	Les validations fonctionnent (erreur si `instance_type` non autorisé)	
4	La connexion SSH à l'instance fonctionne	
5	Le déploiement fonctionne avec deux fichiers `.tfvars` différents (dev + staging)	
6	L'instance est **détruite** en fin	

#	Critère	Vérifié ?
	de séance	
7	Le code est commité, `.tfvars` et `.tfstate` absents de Git, `.tfvars.example` présent	

S05 — Ansible : Premiers pas

Contexte NovaSphere

Le CTO de NovaSphere est satisfait : l'infra se crée en une commande grâce à Terraform. Mais un problème subsiste :

« Terraform crée des serveurs vierges. Qui installe Nginx ? Qui configure le firewall ? Qui déploie l'application ? Actuellement, chaque admin se connecte en SSH et tape des commandes à la main. Ça ne scale pas, et si la VM tombe, on ne sait pas la reconfigurer à l'identique. Il nous faut un outil de ***gestion de configuration****. »*

Objectif du sprint : *Configurer automatiquement un serveur web Nginx sur votre EC2, en une seule commande, de façon reproductible.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Comprendre** le rôle d'Ansible et sa différence avec Terraform.
2. **Créer** un inventaire pour cibler des machines distantes.
3. **Écrire** un playbook qui installe et configure un service.
4. **Vérifier** l'idempotence d'un playbook Ansible.
5. **Utiliser** les modules Ansible avec leur nom complet (FQCN).

Partie théorique

1. Terraform vs Ansible — Complémentaires, pas concurrents

```
Terraform                          Ansible

┌──────────────────────┐           ┌──────────────────────┐
│ PROVISIONING         │           │ CONFIGURATION        │
│                      │           │                      │
```

```
│ Crée l'infra :        │            │ Configure l'infra :   │
│ - Serveurs (EC2)      │──────────→│ - Packages (nginx)    │
│ - Réseau (SG, VPC)    │  IP/SSH    │ - Fichiers (config)   │
│ - DNS, stockage...    │            │ - Services (systemd)  │
│                       │            │ - Users, permissions  │
│ Langage : HCL         │            │ Langage : YAML        │
│ State : oui           │            │ State : non (agentless)│
└───────────────────────┘            └───────────────────────┘
```

Terraform crée les ressources (VM, réseau, DNS). **Ansible** configure ce qui tourne **dessus** (packages, fichiers, services). Les deux se complètent.

2. Ansible — Architecture sans agent

```
Votre poste (Control Node)              Serveur EC2 (Managed Node)

┌──────────────────────────┐            ┌──────────────────────┐
│ ansible-playbook         │            │                      │
│                          │── SSH ──→│  Python (pré-installé│
│ Inventaire (hosts.yml)   │            │  sur Ubuntu)         │
│ Playbook (playbook.yml)  │            │                      │
│                          │←── résultat─│  Exécute les modules  │
└──────────────────────────┘            └──────────────────────┘
```

Aucun agent à installer sur les serveurs. Ansible se connecte en SSH, exécute des modules Python sur la cible, et récupère le résultat. C'est pour ça qu'on a préparé la clé SSH en S4.

3. L'inventaire — La liste des machines

L'inventaire dit à Ansible **quelles machines** cibler et **comment** s'y connecter.

```yaml
# inventory/hosts.yml
all:
  children:
    webservers:
      hosts:
        novasphere-web:
```

```
        ansible_host: 13.38.xx.xx        # IP publique EC2
        ansible_user: ubuntu             # Utilisateur SSH
        ansible_ssh_private_key_file: ~/.ssh/id_rsa
```

Concept	Description
`all`	Groupe racine (contient tous les hôtes)
`children`	Sous-groupes
`webservers`	Un groupe logique (on peut en créer autant qu'on veut)
`hosts`	Les machines du groupe
`ansible_host`	L'adresse IP ou le hostname
`ansible_user`	L'utilisateur SSH
`ansible_ssh_private_key_file`	Le chemin vers la clé privée

4. Le playbook — La recette de configuration

Un playbook est une liste de **plays** (ensembles de tâches) écrits en YAML :

```
---
- name: Configurer le serveur web
  hosts: webservers
  become: true                    # Exécuter en sudo

  tasks:
    - name: Installer Nginx
      ansible.builtin.apt:        # Module + FQCN
        name: nginx
        state: present
        update_cache: true
```

Élément	Rôle
`name`	Description lisible (apparaît dans les logs)
`hosts`	Groupe de l'inventaire ciblé
`become: true`	Exécuter les tâches en `sudo`
`tasks`	Liste ordonnée de tâches
`ansible.builtin.apt`	Module avec son **FQCN** (Fully Qualified

Élément	Rôle
	Collection Name)

Convention : On écrit `ansible.builtin.apt` et jamais juste `apt`. Le FQCN est le standard depuis Ansible 2.10+ — il évite les ambiguïtés entre modules de différentes collections.

5. L'idempotence — Appliquer sans casser

Comme Terraform, Ansible est **idempotent** : exécuter le même playbook 2 fois ne change rien la deuxième fois.

```
1ère exécution :  "Installer nginx"  →  changed    (nginx installé)

2ème exécution :  "Installer nginx"  →  ok          (nginx déjà présent)
```

Le statut `changed` signifie qu'Ansible a modifié quelque chose. Le statut `ok` signifie que l'état souhaité est déjà atteint.

TP — Premier playbook Ansible

Workflow Git :

```
cd ~/novasphere-infra

git switch -c feat/s05-ansible
```

Exercice 0 — Déployer l'EC2 avec Terraform (5 min)

On a besoin d'un serveur pour travailler. Relancez l'infra de S4 :

```
cd ~/novasphere-infra/terraform/aws

terraform init

terraform apply
```

Notez l'IP publique :

```
export EC2_IP=$(terraform output -raw public_ip)

echo "IP de l'instance : $EC2_IP"
```

Vérifiez l'accès SSH :

```
ssh -o StrictHostKeyChecking=no ubuntu@$EC2_IP "hostname"

# Doit afficher le hostname de l'instance
```

Si ça échoue : vérifiez que l'instance est Running dans la console AWS, que le Security Group autorise le port 22, et que votre clé SSH est la bonne.

Exercice 1 — Inventaire et premier test (10 min)

Étape 1 — Créez la structure Ansible :

```
cd ~/novasphere-infra
mkdir -p ansible/inventory
```

Étape 2 — Créez le fichier d'inventaire `ansible/inventory/hosts.yml` :

```
---
all:
  children:
    webservers:
      hosts:
        novasphere-web:
          ansible_host: ___________              # TODO 1 : IP publique de votre EC2
          ansible_user: ubuntu
          ansible_ssh_private_key_file: ~/.ssh/id_rsa
```

Étape 3 — Créez le fichier de configuration `ansible/ansible.cfg` :

```
[defaults]
inventory = inventory/hosts.yml
host_key_checking = false
retry_files_enabled = false

[privilege_escalation]
become = false
```

> **Note** : `host_key_checking = false` — Désactive la vérification de l'empreinte SSH. En production, on le laisse à `true`. Pour le TP, ça évite le prompt « Are you sure you want to continue connecting? » qui bloque Ansible.

Étape 4 — Testez la connexion :

```
cd ~/novasphere-infra/ansible
```

```
# Ping tous les hôtes de l'inventaire
ansible all -m ansible.builtin.ping
```

Résultat attendu :

```
novasphere-web | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

Question : Le module `ping` ne fait pas un ping réseau (ICMP). Que teste-t-il réellement ?

Réponse : _____

Étape 5 — Exécutez une commande ad-hoc :

```
# Afficher la version d'Ubuntu sur la cible
ansible all -m ansible.builtin.command -a "lsb_release -ds"
```

Exercice 2 — Premier playbook : installer Nginx (20 min)

Étape 1 — Créez le fichier `ansible/playbook.yml` :

```
---
- name: Configuration du serveur web NovaSphere
  hosts: ___________                    # TODO 2 : Quel groupe de
l'inventaire ?
  become: true                          # Exécuter en sudo


  tasks:
    # --- Mise à jour du cache APT ---
    - name: Mettre à jour le cache APT
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600          # Ne rafraîchir que si > 1h
```

```yaml
    # --- Installation de Nginx ---
    - name: Installer Nginx
      ansible.builtin.apt:
        name: ___________                # TODO 3 : Quel package installer ?
        state: present

    # --- Vérifier que Nginx tourne ---
    - name: S'assurer que Nginx est démarré et activé au boot
      ansible.builtin.service:
        name: nginx
        state: ___________               # TODO 4 : Quel état pour un service qui doit tourner ?
        enabled: true

    # --- Déployer une page d'accueil personnalisée ---
    - name: Déployer la page d'accueil NovaSphere
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: |
          <!DOCTYPE html>
          <html>
          <head><title>NovaSphere</title></head>
          <body>
            <h1>Bienvenue sur NovaSphere</h1>
            <p>Serveur configuré automatiquement par Ansible.</p>
            <p>Environnement : dev</p>
          </body>
```

```
          </html>
        owner: www-data
        group: www-data
        mode: "0644"
```

Étape 2 — Exécutez le playbook :

```
cd ~/novasphere-infra/ansible
ansible-playbook playbook.yml
```

Étape 3 — Observez la sortie :

```
PLAY [Configuration du serveur web NovaSphere] *********

TASK [Gathering Facts] *********************************
ok: [novasphere-web]

TASK [Mettre à jour le cache APT] **********************
changed: [novasphere-web]

TASK [Installer Nginx] *********************************
changed: [novasphere-web]

TASK [S'assurer que Nginx est démarré et activé au boot]
ok: [novasphere-web]

TASK [Déployer la page d'accueil NovaSphere] ***********
changed: [novasphere-web]

PLAY RECAP *********************************************
novasphere-web  : ok=5  changed=3  unreachable=0  failed=0
```

Question : Pourquoi « S'assurer que Nginx tourne » affiche `ok` et non `changed` ?

Réponse : _____

Étape 4 — Vérifiez le résultat :

Il faut d'abord autoriser le trafic HTTP. Ajoutez temporairement une règle dans le Security Group AWS :

```
# Récupérez l'ID du Security Group
cd ~/novasphere-infra/terraform/aws
SG_ID=$(terraform output -raw security_group_id)

# Ajoutez une règle HTTP (port 80)
aws ec2 authorize-security-group-ingress \
  --group-id $SG_ID \
  --protocol tcp \
  --port 80 \
  --cidr 0.0.0.0/0
```

Maintenant, ouvrez dans votre navigateur :

```
http://<IP_PUBLIQUE>
```

Vous devriez voir la page « Bienvenue sur NovaSphere ».

Exercice 3 — Vérifier l'idempotence (10 min)

Étape 1 — Relancez exactement le même playbook :

```
cd ~/novasphere-infra/ansible
ansible-playbook playbook.yml
```

Étape 2 — Observez :

```
PLAY RECAP *********************************************
novasphere-web  : ok=5  changed=0  unreachable=0  failed=0
```

Question : `changed=0`. Pourquoi ? Qu'est-ce que ça prouve ?

Réponse : _____

Étape 3 — Cassez volontairement l'idempotence :

Modifiez la page HTML dans le playbook (changez le texte), puis relancez :

```
ansible-playbook playbook.yml
```

> **Question** : Quelle tâche passe en `changed` ? Les autres tâches sont-elles affectées ?
>
> Réponse : _____

Exercice 4 — Le handler : redémarrer intelligemment (15 min)

Quand on modifie un fichier de configuration, il faut souvent redémarrer le service. Mais on ne veut pas redémarrer à chaque exécution — seulement quand la configuration **a changé**. C'est le rôle d'un **handler**.

Étape 1 — Améliorez le playbook en ajoutant un handler :

```yaml
---
- name: Configuration du serveur web NovaSphere
  hosts: webservers
  become: true

  tasks:
    - name: Mettre à jour le cache APT
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    - name: Installer Nginx
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: S'assurer que Nginx est démarré et activé
      ansible.builtin.service:
        name: nginx
        state: started
```

```
        enabled: true


    - name: Déployer la page d'accueil NovaSphere
      ansible.builtin.copy:
        dest: /var/www/html/index.html
        content: |
          <!DOCTYPE html>
          <html>
          <head><title>NovaSphere</title></head>
          <body>
            <h1>Bienvenue sur NovaSphere</h1>
            <p>Serveur configuré automatiquement par Ansible.</p>
            <p>Environnement : dev | Version : 2.0</p>
          </body>
          </html>
        owner: www-data
        group: www-data
        mode: "0644"
      notify: ___________                # TODO 5 : Quel handler
notifier ?


  handlers:
    - name: Redémarrer Nginx
      ansible.builtin.service:
        name: nginx
        state: ___________              # TODO 6 : Quel état pour un
redémarrage ?
```

Étape 2 — Exécutez :

```
ansible-playbook playbook.yml
```

Question : Le handler s'est-il exécuté ? Pourquoi ou pourquoi pas ?

Réponse : _____

Étape 3 — Modifiez le contenu HTML (changez « Version : 2.0 » en « Version : 3.0 »), puis relancez :

```
ansible-playbook playbook.yml
```

Question : Cette fois, le handler s'exécute. Pourquoi ?

Réponse : _____

Exercice 5 — Nettoyage et commit (10 min)

Étape 1 — Détruisez l'infra AWS :

```
cd ~/novasphere-infra/terraform/aws

terraform destroy -auto-approve
```

Important : Vérifiez dans la console AWS que l'instance est terminated.

Étape 2 — Committez :

```
cd ~/novasphere-infra


git add ansible/

git commit -m "feat: premier playbook Ansible - installation Nginx"


git switch main

git merge feat/s05-ansible

git branch -d feat/s05-ansible

git tag -a v0.5.0 -m "S05: Ansible premiers pas, Nginx déployé"

git push && git push --tags
```

Critères de réussite

#	Critère	Vérifié ?
1	`ansible all -m ansible.builtin.ping` retourne SUCCESS	

#	Critère	Vérifié ?
2	Le playbook s'exécute sans erreur (`failed=0`)	
3	La page Nginx est accessible dans le navigateur	
4	La 2ème exécution affiche `changed=0` (idempotence)	
5	Le handler se déclenche uniquement quand le HTML change	
6	Tous les modules utilisent le FQCN (`ansible.builtin.*`)	
7	L'instance EC2 est **détruite** en fin de séance	
8	Le code est commité avec le tag `v0.5.0`	

S06 — Ansible avancé : Variables, Boucles & Templates

Contexte NovaSphere

L'équipe DevOps de NovaSphere est satisfaite : le serveur web se configure en une commande grâce à Ansible. Mais le CTO pose une question :

« Si demain je veux installer 10 paquets au lieu de 2, tu vas écrire 10 tâches identiques ? Et si le nom de domaine change entre dev et prod, tu modifies le playbook à chaque déploiement ? Il nous faut des variables, des boucles, et des fichiers de configuration générés dynamiquement. Bref, il faut industrialiser. »

Objectif du sprint : *Un playbook paramétré, réutilisable, qui génère sa configuration à partir de templates.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Externaliser** la configuration d'un playbook dans des fichiers de variables (`vars_files`, `group_vars`).
2. **Utiliser** des boucles (`loop`) pour éviter la duplication de tâches.
3. **Appliquer** des conditions (`when`) pour adapter le comportement selon le contexte.
4. **Créer** des templates Jinja2 pour générer des fichiers de configuration dynamiques.

Partie théorique

1. Les variables — Paramétrer les playbooks

En S05, le nom de domaine, le contenu HTML et le port étaient écrits en dur dans le playbook. C'est exactement le même problème que les valeurs hardcodées en Terraform (S03 → S04).

Les variables Ansible suivent une **hiérarchie de priorité** (de la plus faible à la plus forte) :

```
1. defaults/main.yml (dans un rôle)        ← priorité la plus faible
2. group_vars/all.yml
3. group_vars/<nom_groupe>.yml
4. host_vars/<nom_hôte>.yml
5. vars_files (dans le playbook)
6. vars (dans le playbook)
7. -e / --extra-vars (ligne de commande)    ← priorité la plus forte
```

Exemple :

```yaml
# Dans le playbook
vars:
  app_name: "NovaSphere"
  http_port: 80

# Utilisation dans une tâche
- name: "Afficher le nom de l'application"
  ansible.builtin.debug:
    msg: "Application : {{ app_name }} sur le port {{ http_port }}"
```

> **Convention** : On ne met **jamais** de variables dans le playbook principal en production. On les externalise dans des fichiers dédiés (`vars_files` ou `group_vars`). Le playbook ne contient que la logique, pas les données.

2. Les boucles — Ne pas se répéter

En S05, pour installer Nginx, une seule tâche suffisait. Mais si NovaSphere a besoin de 5 paquets, on ne va pas écrire 5 tâches identiques.

```
# Sans boucle (S05) — une tâche par paquet
- name: Installer Nginx
  ansible.builtin.apt:
    name: nginx
    state: present

- name: Installer curl
  ansible.builtin.apt:
    name: curl
    state: present

# Avec boucle (S06) — une seule tâche
- name: Installer les paquets requis
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - curl
    - htop
    - tree
    - jq
```

La variable spéciale `{{ item }}` prend successivement chaque valeur de la liste.

Boucle sur une variable (meilleure pratique) :

```
vars:
  packages:
```

```yaml
    - nginx
    - curl
    - htop

tasks:
  - name: Installer les paquets requis
    ansible.builtin.apt:
      name: "{{ item }}"
      state: present
    loop: "{{ packages }}"
```

3. Les conditions — Adapter le comportement

Le mot-clé `when` permet d'exécuter une tâche uniquement si une condition est remplie :

```yaml
- name: Installer Apache (si demandé)
  ansible.builtin.apt:
    name: apache2
    state: present
  when: web_server == "apache"

- name: Installer Nginx (si demandé)
  ansible.builtin.apt:
    name: nginx
    state: present
  when: web_server == "nginx"
```

Conditions courantes :

Condition	Signification
`when: ansible_os_family == "Debian"`	Uniquement sur Debian/Ubuntu
`when: env == "prod"`	Uniquement en production
`when: enable_monitoring | bool`	Si la variable est `true`

Condition	Signification
`when: packages is defined`	Si la variable existe

4. Les templates Jinja2 — Générer des fichiers dynamiques

En S05, le HTML était écrit en dur dans le module `ansible.builtin.copy`. C'est fragile et peu flexible.

Un **template Jinja2** est un fichier modèle (extension `.j2`) qui contient des variables et de la logique :

```
{# templates/index.html.j2 #}

<!DOCTYPE html>

<html>

<head><title>{{ app_name }}</title></head>

<body>

  <h1>Bienvenue sur {{ app_name }}</h1>

  <p>Serveur : {{ inventory_hostname }}</p>

  <p>Environnement : {{ env }}</p>

  <p>Version : {{ app_version }}</p>

</body>

</html>
```

On utilise le module `ansible.builtin.template` au lieu de `ansible.builtin.copy` :

```yaml
- name: Déployer la page d'accueil

  ansible.builtin.template:

    src: templates/index.html.j2

    dest: /var/www/html/index.html

    owner: www-data

    group: www-data

    mode: '0644'
```

La différence clé : `copy` envoie un fichier tel quel, `template` le **rend** (remplace les `{{ }}`) avant de l'envoyer.

TP — Industrialiser le playbook NovaSphere

Workflow Git :

```
cd ~/novasphere-infra

git switch -c feat/s06-ansible-avance
```

Exercice 0 — Déployer l'EC2 avec Terraform (5 min)

```
cd ~/novasphere-infra/terraform/aws

terraform apply -auto-approve

# Notez l'IP publique affichée dans les outputs
```

Exercice 1 — Externaliser les variables (15 min)

En S05, tout était dans le playbook. On va séparer les données de la logique.

Étape 1 — Créez la structure de variables :

```
cd ~/novasphere-infra/ansible

mkdir -p group_vars
```

Étape 2 — Créez le fichier `group_vars/webservers.yml` :

```
# group_vars/webservers.yml — Variables du groupe webservers

---

app_name: "NovaSphere"

app_version: "2.0"

env: "dev"

http_port: 80


# Paquets à installer sur les serveurs web

packages:

  - nginx

  - curl

  - htop

  - tree

  - jq
```

```yaml
# Configuration Nginx
nginx_server_name: "_"
nginx_root: "/var/www/html"
```

Étape 3 — Mettez à jour l'inventaire pour utiliser le groupe `webservers` :

```yaml
# inventory/hosts.yml
---
all:
  children:
    webservers:
      hosts:
        web01:
          ansible_host: ___________    # IP publique de votre EC2
          ansible_user: ubuntu
          ansible_ssh_private_key_file: ~/.ssh/novasphere
```

> **Question** : Pourquoi les variables sont dans `group_vars/webservers.yml` et pas `group_vars/all.yml` ?
>
> Réponse : _____

Exercice 2 — Boucles et playbook paramétré (20 min)

Étape 1 — Réécrivez le playbook `playbook.yml` en utilisant les variables et les boucles :

```yaml
# playbook.yml — S06 : Playbook paramétré NovaSphere
---
- name: Configuration du serveur web NovaSphere
  hosts: webservers
  become: true

  tasks:
    # --- Mise à jour du cache APT ---
```

```yaml
    - name: Mettre à jour le cache APT
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600

    # --- Installation des paquets (avec boucle) ---
    - name: "Installer les paquets requis"
      ansible.builtin.apt:
        name: "___________"          # TODO 1 : Quelle variable
spéciale pour la boucle ?
        state: present
      loop: "___________"            # TODO 2 : Quelle variable
contient la liste des paquets ?

    # --- S'assurer que Nginx tourne ---
    - name: S'assurer que Nginx est démarré et activé
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true

    # --- Déployer la configuration Nginx ---
    - name: Déployer la configuration du virtual host
      ansible.builtin.template:
        src: ___________             # TODO 3 : Chemin vers le
template du vhost
        dest: /etc/nginx/sites-available/novasphere.conf
        owner: root
        group: root
        mode: '0644'
```

```
      notify: ___________            # TODO 4 : Quel handler
déclencher ?


    # --- Activer le virtual host ---
    - name: Activer le virtual host NovaSphere
      ansible.builtin.file:
        src: /etc/nginx/sites-available/novasphere.conf
        dest: /etc/nginx/sites-enabled/novasphere.conf
        state: link
      notify: Redémarrer Nginx

    # --- Supprimer le site par défaut ---
    - name: Supprimer le virtual host par défaut
      ansible.builtin.file:
        path: /etc/nginx/sites-enabled/default
        state: absent
      notify: Redémarrer Nginx

    # --- Déployer la page d'accueil ---
    - name: Déployer la page d'accueil NovaSphere
      ansible.builtin.template:
        src: templates/index.html.j2
        dest: "{{ nginx_root }}/index.html"
        owner: www-data
        group: www-data
        mode: '0644'

  handlers:
    - name: Redémarrer Nginx
```

```
      ansible.builtin.service:
        name: nginx
        state: restarted
```

Étape 2 — Créez le répertoire des templates :

```
mkdir -p templates
```

Étape 3 — Créez le template du virtual host `templates/novasphere.conf.j2` :

```
# templates/novasphere.conf.j2 — Généré par Ansible
# NE PAS MODIFIER MANUELLEMENT

server {
    listen {{ http_port }};
    server_name {{ nginx_server_name }};

    root {{ nginx_root }};
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    # Header personnalisé pour identifier l'environnement
    add_header X-Environment "{{ env }}";
    add_header X-Managed-By "Ansible";
}
```

Étape 4 — Créez le template de la page d'accueil `templates/index.html.j2` :

```
{# templates/index.html.j2 — Généré par Ansible #}
<!DOCTYPE html>
<html lang="fr">
```

```
<head>
    <meta charset="UTF-8">
    <title>{{ app_name }} — {{ env }}</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 600px;
margin: 50px auto; }
        h1 { color: #2c3e50; }
        table { border-collapse: collapse; width: 100%; }
        td, th { border: 1px solid #ddd; padding: 8px; text-align:
left; }
        th { background-color: #3498db; color: white; }
    </style>
</head>
<body>
    <h1>{{ app_name }}</h1>
    <table>
        <tr><th>Information</th><th>Valeur</th></tr>
        <tr><td>Environnement</td><td>{{ env }}</td></tr>
        <tr><td>Version</td><td>{{ app_version }}</td></tr>
        <tr><td>Serveur</td><td>{{ inventory_hostname }}</td></tr>
        <tr><td>IP</td><td>{{ ansible_host }}</td></tr>
        <tr><td>Paquets installés</td><td>{{ packages | join(',
') }}</td></tr>
    </table>
    <p><em>Page générée par Ansible — ne pas modifier
manuellement.</em></p>
</body>
</html>
```

Étape 5 — Exécutez :

```
cd ~/novasphere-infra/ansible
```

```
ansible-playbook -i inventory/hosts.yml playbook.yml
```

Étape 6 — Vérifiez dans le navigateur :

Ouvrez `http://<IP_PUBLIQUE>` — vous devriez voir la page NovaSphere avec le tableau d'informations.

> **Question** : Combien de tâches affichent `changed` à la première exécution ? Relancez le playbook — combien maintenant ?
>
> Réponse : _____

Exercice 3 — Conditions et multi-environnement (15 min)

Étape 1 — Ajoutez une condition dans le playbook. Après la tâche « Déployer la page d'accueil », ajoutez :

```
    # --- Installer les outils de debug (dev uniquement) ---
    - name: Installer les outils de debug
      ansible.builtin.apt:
        name: "{{ item }}"
        state: present
      loop:
        - strace
        - tcpdump
        - net-tools
      when: ___________              # TODO 5 : Quelle condition pour
exécuter uniquement en dev ?
```

Étape 2 — Exécutez et observez :

```
ansible-playbook -i inventory/hosts.yml playbook.yml
```

> **Question** : La tâche « Installer les outils de debug » s'exécute-t-elle ? Pourquoi ?
>
> Réponse : _____

Étape 3 — Testez en surchargeant la variable :

```
ansible-playbook -i inventory/hosts.yml playbook.yml -e "env=prod"
```

> **Question** : Cette fois, la tâche debug s'exécute-t-elle ? Que s'est-il passé avec la page HTML (rechargez-la dans le navigateur) ?
>
> Réponse : _____

Étape 4 — Revenez en dev :

```
ansible-playbook -i inventory/hosts.yml playbook.yml
```

> **Note** : L'option `-e` (extra-vars) a la priorité la plus haute. C'est ce qui permet aux pipelines CI/CD de surcharger l'environnement sans modifier de fichier.

Exercice 4 — Boucle sur dictionnaires (10 min)

Les boucles ne se limitent pas aux listes simples. On peut itérer sur des dictionnaires.

Étape 1 — Ajoutez dans `group_vars/webservers.yml` :

```yaml
# Utilisateurs applicatifs à créer
app_users:
  - name: deployer
    comment: "Compte de déploiement NovaSphere"
    shell: /bin/bash
  - name: monitoring
    comment: "Compte de monitoring"
    shell: /usr/sbin/nologin
```

Étape 2 — Ajoutez la tâche dans le playbook (avant les handlers) :

```yaml
    # --- Créer les utilisateurs applicatifs ---
    - name: Créer les utilisateurs applicatifs
      ansible.builtin.user:
        name: "{{ item.name }}"
        comment: "{{ item.comment }}"
        shell: "{{ item.shell }}"
        create_home: true
        state: present
      loop: "{{ app_users }}"
```

Étape 3 — Exécutez et vérifiez :

```
ansible-playbook -i inventory/hosts.yml playbook.yml
```

```
# Vérification sur le serveur

ssh -i ~/.ssh/novasphere ubuntu@<IP> "cat /etc/passwd | grep -E
'deployer|monitoring'"
```

> **Question** : Pourquoi `monitoring` a `/usr/sbin/nologin` comme shell ? Que se passe-t-il si on essaie de se connecter avec ce compte ?
>
> Réponse : _____

Exercice 5 — Nettoyage et commit (5 min)

Étape 1 — Détruisez l'infra AWS :

```
cd ~/novasphere-infra/terraform/aws

terraform destroy -auto-approve
```

> **Important** : Vérifiez dans la console AWS que l'instance est terminated.

Étape 2 — Committez :

```
cd ~/novasphere-infra



git add ansible/

git commit -m "feat: ansible variables, boucles, templates Jinja2"



git switch main

git merge feat/s06-ansible-avance

git branch -d feat/s06-ansible-avance

git tag -a v0.6.0 -m "S06: Ansible avancé - variables, boucles,
templates"

git push && git push --tags
```

Critères de réussite

#	Critère	Vérifié ?
1	Les variables sont externalisées dans `group_vars/webservers.yml`	
2	Les paquets sont installés via	

#	Critère	Vérifié ?
	une boucle `loop` (pas de tâches dupliquées)	
3	La page HTML est générée par un template Jinja2 (pas un `copy`)	
4	Le virtual host Nginx est généré par template	
5	Les outils de debug ne s'installent qu'en `env: dev`	
6	`-e "env=prod"` change le comportement (skip debug + page HTML mise à jour)	
7	Les utilisateurs applicatifs sont créés via une boucle sur dictionnaire	
8	L'instance EC2 est **détruite** en fin de séance	
9	Le code est commité avec le tag `v0.6.0`	

S07 — Ansible : Rôles & Vault

Contexte NovaSphere

Le playbook de NovaSphere fonctionne parfaitement. Mais il fait déjà 80 lignes et ne gère qu'un seul service (Nginx). Quand l'équipe ajoutera une base de données, un service de monitoring et une application backend, le fichier sera ingérable.

Le CTO demande deux choses :

« Premièrement, structurez le code Ansible comme on a structuré Terraform en S04 : un fichier par responsabilité. Deuxièmement, le mot de passe de la base de données ne peut pas trainer en clair dans le dépôt Git. Chiffrez-le. »

Objectif du sprint : *Un rôle réutilisable + des secrets chiffrés. Le code est propre et sécurisé.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Transformer** un playbook monolithique en un rôle structuré (`ansible-galaxy role init`).
2. **Utiliser** les tags Ansible pour exécuter sélectivement des tâches.
3. **Chiffrer** des variables sensibles avec Ansible Vault.
4. **Exécuter** un playbook qui combine rôles et secrets chiffrés.

Partie théorique

1. Les rôles — Structurer le code

Un rôle est une unité de code Ansible **autonome et réutilisable**. C'est l'équivalent d'une fonction en programmation ou d'un module Terraform.

Structure d'un rôle :

```
roles/
└── webserver/
    ├── tasks/
    │   └── main.yml         # Tâches principales
    ├── handlers/
    │   └── main.yml         # Handlers (redémarrages)
    ├── templates/
    │   ├── novasphere.conf.j2
    │   └── index.html.j2
    ├── defaults/
    │   └── main.yml         # Variables par défaut (surchargeables)
    ├── vars/
    │   └── main.yml         # Variables internes (non surchargeables)
    ├── files/
    │   └── (fichiers statiques)
    ├── meta/
    │   └── main.yml         # Métadonnées (dépendances)
    └── README.md
```

Le playbook principal devient minimaliste :

```yaml
# playbook.yml
---
- name: Configuration NovaSphere
  hosts: webservers
  become: true
  roles:
    - webserver
```

Ansible charge automatiquement `roles/webserver/tasks/main.yml`, les handlers de `handlers/main.yml`, les templates de `templates/`, etc. Pas besoin de chemins — la convention suffit.

2. Les tags — Exécution sélective

Les tags permettent de n'exécuter qu'une partie du playbook :

```yaml
- name: Installer Nginx
  ansible.builtin.apt:
    name: nginx
    state: present
  tags: [install, nginx]

- name: Déployer la config
  ansible.builtin.template:
    src: novasphere.conf.j2
    dest: /etc/nginx/sites-available/novasphere.conf
  tags: [config, nginx]
# Exécuter uniquement les tâches taguées "config"
ansible-playbook playbook.yml --tags config

# Tout sauf l'installation
ansible-playbook playbook.yml --skip-tags install
```

```
# Lister les tags disponibles
ansible-playbook playbook.yml --list-tags
```

3. Ansible Vault — Chiffrer les secrets

Ansible Vault chiffre des fichiers ou des variables individuelles avec AES-256 :

```
# Créer un fichier chiffré
ansible-vault create secrets.yml

# Chiffrer un fichier existant
ansible-vault encrypt vars/secrets.yml

# Modifier un fichier chiffré
ansible-vault edit vars/secrets.yml

# Déchiffrer (pour lecture)
ansible-vault view vars/secrets.yml
```

Pour exécuter un playbook qui utilise des fichiers chiffrés :

```
# Demander le mot de passe
ansible-playbook playbook.yml --ask-vault-pass

# Utiliser un fichier mot de passe (CI/CD)
ansible-playbook playbook.yml --vault-password-file ~/.vault_pass
```

> **Convention** : Le fichier `.vault_pass` est dans le `.gitignore` — jamais commité. En CI/CD, le mot de passe Vault est injecté via une variable d'environnement.

TP — Structurer et sécuriser NovaSphere

Workflow Git :

```
cd ~/novasphere-infra
git switch -c feat/s07-roles-vault
```

Exercice 0 — Déployer l'EC2 avec Terraform (5 min)

```
cd ~/novasphere-infra/terraform/aws

terraform apply -auto-approve

# Notez l'IP publique — mettez à jour inventory/hosts.yml
```

Exercice 1 — Créer le rôle `webserver` (25 min)

Étape 1 — Initialisez le rôle avec `ansible-galaxy` :

```
cd ~/novasphere-infra/ansible

mkdir -p roles

ansible-galaxy role init roles/webserver
```

> **Question** : Listez le contenu de `roles/webserver/`. Combien de répertoires ont été créés ? À quoi sert chacun ?
>
> Réponse : _____

Étape 2 — Déplacez les tâches. Éditez `roles/webserver/tasks/main.yml` :

```yaml
# roles/webserver/tasks/main.yml

---

- name: Mettre à jour le cache APT

  ansible.builtin.apt:

    update_cache: true

    cache_valid_time: 3600

  tags: [install]


- name: Installer les paquets requis

  ansible.builtin.apt:

    name: "{{ item }}"

    state: present

  loop: "{{ packages }}"

  tags: [install]
```

```yaml
- name: S'assurer que Nginx est démarré et activé
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
  tags: [config]

- name: Déployer la configuration du virtual host
  ansible.builtin.template:
    src: novasphere.conf.j2
    dest: /etc/nginx/sites-available/novasphere.conf
    owner: root
    group: root
    mode: '0644'
  notify: Redémarrer Nginx
  tags: [config, nginx]

- name: Activer le virtual host NovaSphere
  ansible.builtin.file:
    src: /etc/nginx/sites-available/novasphere.conf
    dest: /etc/nginx/sites-enabled/novasphere.conf
    state: link
  notify: Redémarrer Nginx
  tags: [config, nginx]

- name: Supprimer le virtual host par défaut
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
```

```yaml
    state: absent
  notify: Redémarrer Nginx
  tags: [config, nginx]

- name: Déployer la page d'accueil NovaSphere
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ nginx_root }}/index.html"
    owner: www-data
    group: www-data
    mode: '0644'
  tags: [deploy]

- name: Installer les outils de debug
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop:
    - strace
    - tcpdump
    - net-tools
  when: env == "dev"
  tags: [debug]

- name: Créer les utilisateurs applicatifs
  ansible.builtin.user:
    name: "{{ item.name }}"
    comment: "{{ item.comment }}"
```

```
    shell: "{{ item.shell }}"
    create_home: true
    state: present
  loop: "{{ app_users }}"
  tags: [users]
```

> **Note** : Dans un rôle, les chemins des templates sont relatifs à `roles/<rôle>/templates/`. Plus besoin d'écrire `templates/novasphere.conf.j2` — juste `novasphere.conf.j2` suffit.

Étape 3 — Déplacez les handlers. Éditez `roles/webserver/handlers/main.yml` :

```
# roles/webserver/handlers/main.yml
---
- name: Redémarrer Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

Étape 4 — Déplacez les templates :

```
# Copiez les templates de S06 dans le rôle
cp templates/novasphere.conf.j2 roles/webserver/templates/
cp templates/index.html.j2 roles/webserver/templates/
```

Étape 5 — Déplacez les variables par défaut. Éditez `roles/webserver/defaults/main.yml` :

```
# roles/webserver/defaults/main.yml — Variables par défaut
(surchargeables)
---
app_name: "NovaSphere"
app_version: "2.0"
env: "dev"
http_port: 80


packages:
```

```yaml
  - nginx
  - curl
  - htop
  - tree
  - jq

nginx_server_name: "_"
nginx_root: "/var/www/html"

app_users:
  - name: deployer
    comment: "Compte de déploiement NovaSphere"
    shell: /bin/bash
  - name: monitoring
    comment: "Compte de monitoring"
    shell: /usr/sbin/nologin
```

Étape 6 — Simplifiez le playbook principal. Remplacez tout le contenu de `playbook.yml` :

```yaml
# playbook.yml — S07 : Playbook avec rôle
---
- name: Configuration du serveur web NovaSphere
  hosts: webservers
  become: true
  roles:
    - role: webserver
```

Étape 7 — Testez :

```
ansible-playbook -i inventory/hosts.yml playbook.yml
```

> **Question** : Le résultat est-il identique à S06 ? Le playbook fait maintenant combien de lignes ?
>
> Réponse : _____

Étape 8 — Testez les tags :

```
# Lister tous les tags disponibles
ansible-playbook -i inventory/hosts.yml playbook.yml --list-tags

# Exécuter uniquement le déploiement de la page
ansible-playbook -i inventory/hosts.yml playbook.yml --tags deploy

# Tout sauf le debug
ansible-playbook -i inventory/hosts.yml playbook.yml --skip-tags debug
```

> **Question** : Avec `--tags deploy`, combien de tâches s'exécutent ? Lesquelles sont skippées ?
>
> Réponse : _____

Exercice 2 — Ansible Vault : chiffrer les secrets (20 min)

NovaSphere a besoin de stocker un mot de passe de base de données. Ce mot de passe ne doit **jamais** apparaître en clair dans Git.

Étape 1 — Créez un fichier de mot de passe Vault (local, jamais commité) :

```
echo "novasphere2026" > ~/.vault_pass
chmod 600 ~/.vault_pass
```

Étape 2 — Vérifiez que `.vault_pass` est dans le `.gitignore` global :

```
# Dans ~/novasphere-infra/.gitignore, vérifiez la présence de :
# .vault_pass
# Si absent, ajoutez-le
echo ".vault_pass" >> ~/novasphere-infra/.gitignore
```

Étape 3 — Créez un fichier de variables sensibles :

```
cd ~/novasphere-infra/ansible
mkdir -p vault
```

Créez `vault/secrets.yml` avec le contenu suivant :

```
# vault/secrets.yml — Secrets NovaSphere (sera chiffré)
```

```yaml
---
db_password: "S3cret_N0vaSph3re!"
api_key: "ns-api-key-2026-dev-abc123"
```

Étape 4 — Chiffrez le fichier :

```
ansible-vault encrypt vault/secrets.yml --vault-password-file
~/.vault_pass
```

Étape 5 — Vérifiez que le fichier est bien chiffré :

```
cat vault/secrets.yml
# Doit afficher : $ANSIBLE_VAULT;1.1;AES256 suivi de données chiffrées


# Pour voir le contenu déchiffré :
ansible-vault view vault/secrets.yml --vault-password-file
~/.vault_pass
```

Étape 6 — Utilisez les secrets dans le playbook. Modifiez `playbook.yml` :

```yaml
# playbook.yml — S07 : Rôle + Vault
---
- name: Configuration du serveur web NovaSphere
  hosts: webservers
  become: true

  vars_files:
    - vault/secrets.yml

  roles:
    - role: webserver

  tasks:
    - name: Afficher que les secrets sont chargés (sans les révéler)
      ansible.builtin.debug:
```

```
        msg: "Le mot de passe DB fait {{ db_password | length }}
caractères"

      tags: [vault-test]
```

Étape 7 — Exécutez avec le Vault :

```
ansible-playbook -i inventory/hosts.yml playbook.yml \

  --vault-password-file ~/.vault_pass
```

> **Question** : Le debug affiche combien de caractères pour le mot de passe ? Le mot de passe lui-même apparaît-il dans la sortie ?
>
> Réponse : _____

Étape 8 — Modifiez un secret :

```
ansible-vault edit vault/secrets.yml --vault-password-file
~/.vault_pass

# Changez la version de l'api_key, sauvegardez
```

> **Note** : `ansible-vault edit` ouvre le fichier déchiffré dans votre éditeur (`$EDITOR`), puis le re-chiffre à la sauvegarde. Le fichier sur disque reste toujours chiffré.

Exercice 3 — Intégrer les secrets dans le template (10 min)

Étape 1 — Modifiez le template `roles/webserver/templates/index.html.j2`. Ajoutez dans le tableau, avant la ligne `</table>` :

```
        <tr><td>API Key (masquée)</td><td>{{ api_key[:8]
}}********</td></tr>

        <tr><td>DB configurée</td><td>Oui ({{ db_password | length }}
chars)</td></tr>
```

Étape 2 — Relancez le playbook et vérifiez dans le navigateur :

```
ansible-playbook -i inventory/hosts.yml playbook.yml \

  --vault-password-file ~/.vault_pass
```

> **Question** : La page affiche-t-elle le mot de passe complet ou la version masquée ? Que se passe-t-il si vous oubliez `--vault-password-file` ?
>
> Réponse : _____

Exercice 4 — Nettoyage et commit (10 min)

Étape 1 — Nettoyez les anciens fichiers de S06 devenus inutiles :

```
cd ~/novasphere-infra/ansible

# Les templates sont maintenant dans le rôle
rm -rf templates/

# Les variables sont dans defaults du rôle
# group_vars/ reste pour les surcharges par environnement
```

Étape 2 — Vérifiez que le fichier Vault est bien chiffré avant de commiter :

```
head -1 vault/secrets.yml
# DOIT afficher : $ANSIBLE_VAULT;1.1;AES256
# Si vous voyez du YAML en clair → STOP, chiffrez d'abord !
```

Étape 3 — Détruisez l'infra AWS :

```
cd ~/novasphere-infra/terraform/aws
terraform destroy -auto-approve
```

> **Important** : Vérifiez dans la console AWS que l'instance est terminated.

Étape 4 — Committez :

```
cd ~/novasphere-infra

git add ansible/ .gitignore
git commit -m "feat: rôle webserver + Ansible Vault pour les secrets"

git switch main
git merge feat/s07-roles-vault
git branch -d feat/s07-roles-vault
git tag -a v0.7.0 -m "S07: rôles Ansible + Vault"
git push && git push --tags
```

Critères de réussite

#	Critère	Vérifié ?
1	Le rôle `webserver` existe dans `roles/webserver/` avec la bonne structure	
2	Le playbook principal fait moins de 15 lignes	
3	`--tags deploy` n'exécute que le déploiement de la page HTML	
4	`vault/secrets.yml` est chiffré (commence par `$ANSIBLE_VAULT`)	
5	Le playbook s'exécute avec `--vault-password-file` sans erreur	
6	Le mot de passe n'apparaît jamais en clair dans la sortie Ansible	
7	La page HTML affiche l'API key masquée et la longueur du mot de passe	
8	`.vault_pass` n'est PAS dans le dépôt Git	
9	L'instance EC2 est **détruite** en fin de séance	
10	Le code est commité avec le tag `v0.7.0`	

S09 — Automatisation : Terraform → Ansible

Contexte NovaSphere

Depuis S05, chaque déploiement suit le même rituel : `terraform apply`, copier l'IP publique, coller dans l'inventaire Ansible, lancer le playbook. En S08, vous avez documenté ce pipeline — et vous avez sûrement remarqué le maillon faible : la copie manuelle de l'IP.

Le CTO en a assez :

> *« Si un développeur doit copier-coller une IP à chaque déploiement, on n'est pas dans du vrai IaC. Je veux que Terraform génère l'inventaire Ansible automatiquement. Et tant qu'on y est, montrez-moi comment on fait en entreprise avec un inventaire dynamique. »*
>
> **Objectif du sprint** : *Zéro copier-coller. Terraform crée l'infra ET l'inventaire. Ansible se lance sans intervention manuelle.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Générer** un inventaire Ansible automatiquement depuis les outputs Terraform (`local_file`).
2. **Chaîner** `terraform apply` et `ansible-playbook` dans un script unique.
3. **Configurer** un inventaire dynamique AWS avec le plugin `amazon.aws.aws_ec2`.
4. **Comparer** les approches statique générée vs dynamique.

Partie théorique

1. Le problème : le copier-coller

Depuis S05, à chaque `terraform apply`, l'IP change. Vous devez manuellement : 1. Lire l'output `public_ip` 2. Ouvrir `inventory/hosts.yml` 3. Remplacer l'ancienne IP 4. Lancer Ansible

C'est fragile (erreur de frappe), lent, et ça ne scale pas (imaginez 20 serveurs).

2. Solution 1 : Inventaire généré par Terraform

Terraform peut créer des fichiers avec la ressource `local_file`. On lui fait écrire l'inventaire Ansible :

```
resource "local_file" "ansible_inventory" {

  content = templatefile("${path.module}/templates/inventory.tftpl", {

    web_ip   = aws_instance.web.public_ip

    ssh_user = "ubuntu"

    ssh_key  = var.ssh_private_key_path

  })

  filename =
"${path.module}/../../ansible/inventory/hosts_generated.yml"

}
```

Après `terraform apply`, le fichier `hosts_generated.yml` est créé automatiquement avec la bonne IP.

3. Solution 2 : Inventaire dynamique AWS

Au lieu de générer un fichier, Ansible interroge directement l'API AWS pour trouver les instances :

```
# inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions:
  - eu-west-3
filters:
  tag:Project: NovaSphere
  instance-state-name: running
```

Ansible découvre les instances **en temps réel**. Pas de fichier à générer, pas d'IP à copier. C'est l'approche standard en entreprise.

4. Comparaison

Critère	Inventaire généré	Inventaire dynamique
Dépendance Terraform	Oui (fichier généré par TF)	Non (Ansible interroge AWS directement)
Temps réel	Non (snapshot au moment de l'apply)	Oui (requête API à chaque exécution)
Multi-provider	Oui (tout infra supportée)	Dépend du plugin (AWS, Azure, GCP…)
Complexité	Faible	Moyenne (plugin + filtre)
Utilisation courante	Petites infras, démos	Production, infras dynamiques

TP — Automatiser le pipeline

Workflow Git :

```
cd ~/novasphere-infra
git switch -c feat/s09-automation
```

Exercice 1 — Inventaire généré par Terraform (25 min)

Étape 1 — Créez le template d'inventaire. Dans `terraform/aws/`, créez un répertoire `templates/` :

```
cd ~/novasphere-infra/terraform/aws
mkdir -p templates
```

Créez `templates/inventory.tftpl` :

```
# Généré automatiquement par Terraform — NE PAS MODIFIER MANUELLEMENT
---
all:
  children:
    webservers:
      hosts:
        web01:
          ansible_host: ${web_ip}
          ansible_user: ${ssh_user}
          ansible_ssh_private_key_file: ${ssh_key}
```

Étape 2 — Ajoutez la variable `ssh_private_key_path` dans `variables.tf` :

```
variable "ssh_private_key_path" {
  description = "Chemin vers la clé privée SSH pour Ansible"
  type        = string
  default     = "~/.ssh/novasphere"
}
```

Étape 3 — Ajoutez la ressource `local_file` dans `main.tf` :

```
# --- Génération de l'inventaire Ansible ---
resource "local_file" "ansible_inventory" {
  content = templatefile("${path.module}/templates/inventory.tftpl", {
    web_ip   = ___________                    # TODO 1 : Attribut IP
publique de l'instance
    ssh_user = "ubuntu"
    ssh_key  = var.ssh_private_key_path
  })
```

```
  filename        =
"${path.module}/../../ansible/inventory/hosts_generated.yml"

  file_permission = "0644"

}
```

Étape 4 — Ajoutez un output pour confirmer la génération :

```
output "ansible_inventory_path" {

  description = "Chemin de l'inventaire Ansible généré"

  value       = local_file.ansible_inventory.filename

}
```

Étape 5 — Déployez et vérifiez :

```
terraform apply -auto-approve



# Vérifiez que l'inventaire a été généré

cat ../../ansible/inventory/hosts_generated.yml
```

> **Question** : L'IP dans le fichier généré correspond-elle à l'output `public_ip` ? Que se passe-t-il si vous relancez `terraform apply` après un `destroy` + `apply` ?
>
> Réponse : _____

Étape 6 — Testez avec Ansible :

```
cd ~/novasphere-infra/ansible



# Attendez 30 secondes que l'EC2 finisse de démarrer

sleep 30



ansible-playbook -i inventory/hosts_generated.yml playbook.yml \

  --vault-password-file ~/.vault_pass
```

> **Note** : On utilise `hosts_generated.yml` au lieu de `hosts.yml`. L'ancien fichier reste comme backup.

Exercice 2 — Script de déploiement automatique (15 min)

Étape 1 — Créez un script `deploy.sh` à la racine du dépôt :

```bash
#!/usr/bin/env bash
# deploy.sh — Pipeline de déploiement NovaSphere
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
TF_DIR="${SCRIPT_DIR}/terraform/aws"
ANSIBLE_DIR="${SCRIPT_DIR}/ansible"
INVENTORY="${ANSIBLE_DIR}/inventory/hosts_generated.yml"

echo "=== 1/4 — Provisioning (Terraform) ==="
cd "${TF_DIR}"
terraform init -input=false
terraform apply -auto-approve

echo ""
echo "=== 2/4 — Attente du démarrage EC2 (30s) ==="
sleep 30

echo ""
echo "=== 3/4 — Configuration (Ansible) ==="
cd "${ANSIBLE_DIR}"
ansible-playbook -i "${INVENTORY}" playbook.yml \
  --vault-password-file ~/.vault_pass

echo ""
echo "=== 4/4 — Vérification ===="
PUBLIC_IP=$(cd "${TF_DIR}" && terraform output -raw public_ip)
echo "Page NovaSphere : http://${PUBLIC_IP}"
```

```
curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" "http://$
{PUBLIC_IP}"


echo ""
echo "Pipeline terminé avec succès."
chmod +x deploy.sh
```

Étape 2 — Créez aussi un script de destruction `destroy.sh` :

```
#!/usr/bin/env bash
# destroy.sh — Destruction de l'infrastructure NovaSphere
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
TF_DIR="${SCRIPT_DIR}/terraform/aws"

echo "=== Destruction de l'infrastructure ==="
cd "${TF_DIR}"
terraform destroy -auto-approve

echo "Infrastructure détruite."
chmod +x destroy.sh
```

Étape 3 — Testez le pipeline complet :

```
cd ~/novasphere-infra

# Détruire d'abord si une EC2 tourne encore
./destroy.sh

# Pipeline complet de zéro
./deploy.sh
```

Question : Combien de temps prend le pipeline complet (de `./deploy.sh` à « Pipeline terminé ») ?

Réponse : _____

Exercice 3 — Inventaire dynamique AWS (20 min)

L'inventaire généré résout le copier-coller, mais il reste un fichier statique. L'inventaire dynamique est l'approche production.

Étape 1 — Installez la collection AWS pour Ansible :

```
ansible-galaxy collection install amazon.aws

pip install boto3 botocore --break-system-packages
```

Étape 2 — Créez le fichier d'inventaire dynamique `ansible/inventory/aws_ec2.yml` :

```yaml
# inventory/aws_ec2.yml — Inventaire dynamique AWS

---

plugin: amazon.aws.aws_ec2

regions:

  - eu-west-3


filters:

  tag:Project: NovaSphere

  instance-state-name: running


hostnames:

  - ip-address


compose:

  ansible_host: public_ip_address

  ansible_user: "'ubuntu'"

  ansible_ssh_private_key_file: "'~/.ssh/novasphere'"
```

```yaml
keyed_groups:
  - key: tags.Session
    prefix: session
  - key: tags.Name
    prefix: name
```

> **Note** : Le nom du fichier **doit** se terminer par `aws_ec2.yml` ou `aws_ec2.yaml` pour qu'Ansible reconnaisse le plugin.

Étape 3 — Testez l'inventaire dynamique :

```bash
cd ~/novasphere-infra/ansible


# Lister les hôtes découverts
ansible-inventory -i inventory/aws_ec2.yml --list


# Afficher l'arborescence
ansible-inventory -i inventory/aws_ec2.yml --graph
```

> **Question** : Combien d'hôtes sont découverts ? Dans quels groupes automatiques sont-ils classés ?
>
> Réponse : _____

Étape 4 — Exécutez le playbook avec l'inventaire dynamique :

```bash
ansible-playbook -i inventory/aws_ec2.yml playbook.yml \
  --vault-password-file ~/.vault_pass
```

> **Question** : Le playbook fonctionne-t-il ? Si non, quel est le problème le plus probable ?
>
> Réponse : _____
>
> **Indice** : Vérifiez que le groupe `webservers` dans le playbook correspond aux groupes générés par l'inventaire dynamique. Vous devrez peut-être ajuster `hosts:` dans le playbook ou ajouter un mapping dans le fichier d'inventaire dynamique.

Étape 5 — Ajoutez un groupe explicite. Modifiez `aws_ec2.yml` :

```yaml
# Ajouter après keyed_groups:
groups:
  webservers: "'NovaSphere' in tags.Project"
```

Retestez :

```
ansible-inventory -i inventory/aws_ec2.yml --graph
# Le groupe "webservers" doit apparaître


ansible-playbook -i inventory/aws_ec2.yml playbook.yml \
  --vault-password-file ~/.vault_pass
```

Exercice 4 — Nettoyage et commit (10 min)

```
cd ~/novasphere-infra



./destroy.sh



git add .
git commit -m "feat: inventaire Ansible auto-généré + dynamique AWS +
scripts deploy/destroy"



git switch main
git merge feat/s09-automation
git branch -d feat/s09-automation
git tag -a v0.9.0 -m "S09: automatisation TF→Ansible"
git push && git push --tags
```

Critères de réussite

#	Critère	Vérifié ?
1	`terraform apply` génère automatiquement `hosts_generated.yml`	
2	L'IP dans le fichier généré est correcte	
3	`./deploy.sh` enchaîne TF + Ansible sans intervention	

#	Critère	Vérifié ?
4	`ansible-inventory -i inventory/aws_ec2.yml --list` découvre l'instance	
5	Le playbook fonctionne avec l'inventaire dynamique	
6	L'instance EC2 est **détruite** en fin de séance	
7	Le code est commité avec le tag `v0.9.0`	

S10 — Packer : Images AMI personnalisées

Contexte NovaSphere

Le pipeline fonctionne : Terraform crée le serveur, Ansible le configure. Mais le CTO a chronométré : Ansible met 60 secondes à installer les paquets et configurer Nginx à chaque déploiement. Sur 20 serveurs, ça fait 20 minutes d'attente.

« Et si on pré-cuisinait l'image ? Au lieu d'installer Nginx à chaque fois, on crée une image AMI qui contient déjà tout. Terraform déploie cette image — le serveur est opérationnel en 30 secondes au lieu de 90. C'est ça, l'approche immutable infrastructure. »

Objectif du sprint : *Construire une AMI NovaSphere avec Packer + Ansible, puis la déployer avec Terraform.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Expliquer** le concept d'infrastructure immutable et ses avantages.
2. **Écrire** un template Packer HCL pour construire une AMI AWS.
3. **Utiliser** Ansible comme provisioner Packer pour configurer l'image.
4. **Déployer** une instance EC2 à partir de l'AMI personnalisée.

Partie théorique

1. Infrastructure mutable vs immutable

Jusqu'ici, notre approche est **mutable** : on crée un serveur vierge, puis on le modifie avec Ansible. Si on veut mettre à jour, on re-exécute Ansible sur le serveur existant.

L'approche **immutable** est différente : on ne modifie jamais un serveur en production. Pour mettre à jour, on construit une nouvelle image et on remplace le serveur.

```
Mutable (S05-S09) :

  Serveur vierge → Ansible → Serveur configuré → Ansible (mise à jour)
→ Même serveur



Immutable (S10) :

  Packer + Ansible → Image AMI → Terraform → Serveur prêt

  Mise à jour ? → Nouvelle image → Nouveau serveur → Détruire l'ancien
```

2. Packer — Construire des images

Packer automatise la création d'images machine. Le workflow :

```
  ┌──────────┐      ┌──────────────┐      ┌──────────────┐
┌──────────┐

  │ Template  │────►│ EC2 temporaire│────►│ Provisioners │────►│ AMI
│

  │ HCL      │      │ (build)       │      │ (Ansible)    │      │ créée
│

  └──────────┘      └──────────────┘      └──────────────┘
└──────────┘

       │                  │                    │                    │

   Vous écrivez     Packer lance        Ansible configure    Packer
snapshote

   la recette       une instance        l'instance           et
détruit l'instance

                    temporaire          temporaire
temporaire
```

Packer crée une EC2 temporaire, exécute les provisioners (Ansible), prend un snapshot (AMI), puis détruit l'instance temporaire. Vous ne payez que les quelques minutes de build.

3. Syntaxe Packer HCL

Packer utilise le même langage HCL que Terraform :

```
# Source : d'où part-on (quelle AMI de base, quel type d'instance)
source "amazon-ebs" "ubuntu" {
  ami_name      = "novasphere-web-{{timestamp}}"
  instance_type = "t3.micro"
  region        = "eu-west-3"
  source_ami_filter { ... }
  ssh_username  = "ubuntu"
}

# Build : que fait-on sur l'image
build {
  sources = ["source.amazon-ebs.ubuntu"]

  provisioner "ansible" {
    playbook_file = "../ansible/playbook.yml"
  }
}
```

TP — Construire et déployer une AMI NovaSphere

Workflow Git :

```
cd ~/novasphere-infra
git switch -c feat/s10-packer
```

Exercice 1 — Premier template Packer (25 min)

Étape 1 — Vérifiez que Packer est installé :

```
packer version
# Si absent : sudo apt install -y packer (déjà dans le dépôt HashiCorp ajouté en S01)
```

Étape 2 — Créez la structure :

```
cd ~/novasphere-infra
mkdir -p packer
```

Étape 3 — Créez `packer/novasphere-web.pkr.hcl` :

```
# novasphere-web.pkr.hcl — Image AMI NovaSphere


packer {
  required_plugins {
    amazon = {
      version = ">= 1.3.0"
      source  = "github.com/hashicorp/amazon"
    }
    ansible = {
      version = ">= 1.1.0"
      source  = "github.com/hashicorp/ansible"
    }
  }
}


# --- Variables ---
variable "aws_region" {
  type    = string
  default = "eu-west-3"
}


variable "instance_type" {
  type    = string
  default = "t3.micro"
```

```
}

variable "app_version" {
  type    = string
  default = "2.0"
}

# --- Source : AMI Ubuntu de base ---
source "amazon-ebs" "novasphere" {
  ami_name      = "novasphere-web-${var.app_version}-{{timestamp}}"
  instance_type = var.instance_type
  region        = var.aws_region

  source_ami_filter {
    filters = {
      name                = "ubuntu/images/hvm-ssd-gp3/ubuntu-noble-
24.04-amd64-server-*"
      root-device-type    = "ebs"
      virtualization-type = "hvm"
    }
    most_recent = true
    owners      = ["___________"]    # TODO 1 : ID du propriétaire des
AMI Ubuntu (Canonical)
  }

  ssh_username = "ubuntu"

  tags = {
    Name        = "novasphere-web"
```

```
    Project     = "NovaSphere"
    Version     = var.app_version
    Builder     = "Packer"
    Base_AMI    = "{{ .SourceAMI }}"
    Build_Date  = "{{ .BuildDate }}"
  }
}

# --- Build : provisioning avec Ansible ---
build {
  sources = ["source.amazon-ebs.novasphere"]

  # Provisioner Ansible
  provisioner "ansible" {
    playbook_file = "../ansible/packer-playbook.yml"
    extra_arguments = [
      "--extra-vars", "env=prod app_version=${var.app_version}",
      "--skip-tags", "debug"
    ]
  }
}
```

Étape 4 — Créez un playbook simplifié pour Packer. Créez `ansible/packer-playbook.yml` :

```
# packer-playbook.yml — Playbook pour le build Packer (sans Vault)
---
- name: Configuration de l'image NovaSphere
  hosts: all
  become: true
```

```yaml
vars:
  app_name: "NovaSphere"
  app_version: "2.0"
  env: "prod"
  http_port: 80
  packages:
    - nginx
    - curl
    - htop
    - tree
    - jq
  nginx_server_name: "_"
  nginx_root: "/var/www/html"

tasks:
  - name: Mettre à jour le cache APT
    ansible.builtin.apt:
      update_cache: true

  - name: Installer les paquets requis
    ansible.builtin.apt:
      name: "{{ item }}"
      state: present
    loop: "{{ packages }}"

  - name: S'assurer que Nginx est démarré et activé
    ansible.builtin.service:
      name: nginx
```

```
      state: started
      enabled: true

  - name: Déployer la page d'accueil
    ansible.builtin.copy:
      content: |
        <!DOCTYPE html>
        <html><head><title>{{ app_name }}</title></head>
        <body>
          <h1>{{ app_name }}</h1>
          <p>Environnement : {{ env }}</p>
          <p>Version : {{ app_version }}</p>
          <p>Image construite par Packer</p>
        </body></html>
      dest: "{{ nginx_root }}/index.html"
      owner: www-data
      group: www-data
      mode: '0644'
```

> **Note** : Ce playbook n'utilise pas le rôle `webserver` ni Vault. Packer construit l'image dans un contexte isolé (pas d'inventaire, pas de Vault). On simplifie volontairement pour que le build soit autonome.

Étape 5 — Initialisez et validez :

```
cd ~/novasphere-infra/packer


packer init novasphere-web.pkr.hcl
packer validate novasphere-web.pkr.hcl
```

Étape 6 — Construisez l'AMI :

```
packer build novasphere-web.pkr.hcl
```

> **Note** : Le build prend 3 à 5 minutes. Packer lance une EC2 temporaire, exécute Ansible dessus, prend un snapshot, puis détruit l'instance. Vous ne payez que le temps de build.
>
> **Question** : À la fin du build, Packer affiche l'ID de l'AMI créée (format `ami-xxxxxxxxx`). Notez-le : _____

Exercice 2 — Déployer avec l'AMI custom (20 min)

Étape 1 — Modifiez `terraform/aws/main.tf` pour utiliser l'AMI Packer au lieu de chercher dynamiquement :

Ajoutez une variable dans `variables.tf` :

```
variable "custom_ami_id" {

  description = "ID de l'AMI custom Packer (laisser vide pour utiliser l'AMI Ubuntu de base)"

  type        = string

  default     = ""

}
```

Étape 2 — Modifiez la ressource `aws_instance` dans `main.tf` :

```
resource "aws_instance" "web" {

  ami                    = var.custom_ami_id != "" ? var.custom_ami_id : data.aws_ami.ubuntu.id

  instance_type          = var.instance_type

  key_name               = aws_key_pair.novasphere.key_name

  vpc_security_group_ids = [aws_security_group.novasphere_ssh.id]



  tags = merge(local.common_tags, {

    Name = "${local.name_prefix}-web"

  })

}
```

Étape 3 — Déployez avec l'AMI Packer :

```
cd ~/novasphere-infra/terraform/aws

terraform apply -var="custom_ami_id=___________" -auto-approve
```

```
# Remplacez par l'AMI ID noté à l'exercice 1
```

Étape 4 — Vérifiez :

```
# L'IP publique
terraform output public_ip


# Testez dans le navigateur
curl http://$(terraform output -raw public_ip)
```

> **Question** : Le serveur est-il opérationnel immédiatement (sans exécuter Ansible) ? Quelle est la différence de temps par rapport au pipeline S09 ?
>
> Réponse : _____

Étape 5 — Ajoutez le port HTTP au Security Group. Dans `main.tf`, ajoutez un bloc `ingress` :

```
  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
terraform apply -var="custom_ami_id=<votre-ami-id>" -auto-approve
```

Exercice 3 — Nettoyage (10 min)

Étape 1 — Détruisez l'infra :

```
cd ~/novasphere-infra/terraform/aws
terraform destroy -auto-approve
```

Étape 2 — Supprimez l'AMI (pour éviter les coûts de stockage) :

```
# Trouver l'AMI
aws ec2 describe-images --owners self --query "Images[*].
{ID:ImageId,Name:Name}" --output table
```

```
# Supprimer (remplacez par votre AMI ID)
aws ec2 deregister-image --image-id ami-xxxxxxxxx


# Supprimer le snapshot associé
aws ec2 describe-snapshots --owner-ids self --query "Snapshots[*].
{ID:SnapshotId,Desc:Description}" --output table
aws ec2 delete-snapshot --snapshot-id snap-xxxxxxxxx
```

Étape 3 — Committez :

```
cd ~/novasphere-infra


git add .
git commit -m "feat: Packer AMI NovaSphere + déploiement image custom"


git switch main
git merge feat/s10-packer
git branch -d feat/s10-packer
git tag -a v0.10.0 -m "S10: Packer AMI custom"
git push && git push --tags
```

Critères de réussite

#	Critère	Vérifié ?
1	`packer validate` passe sans erreur	
2	`packer build` crée une AMI (ID notée)	
3	Une EC2 déployée depuis l'AMI custom est opérationnelle sans Ansible	
4	La page HTML affiche « Image construite par Packer »	
5	L'AMI et les snapshots sont supprimés en fin de séance	

#	Critère	Vérifié ?
6	L'instance EC2 est **détruite**	
7	Le code est commité avec le tag `v0.10.0`	

S11 — Infrastructure multi-tier : VPC, Subnets & Bastion

Contexte NovaSphere

Jusqu'ici, toutes les instances NovaSphere sont directement exposées sur internet avec une IP publique. Le RSSI (Responsable Sécurité) tire la sonnette d'alarme :

« Un serveur web en frontal, OK, il doit être accessible. Mais quand on ajoutera la base de données, il est hors de question qu'elle ait une IP publique. On veut une architecture classique : un réseau public pour le web, un réseau privé pour les données, et un bastion comme unique point d'entrée SSH. »

Objectif du sprint : *Déployer une infrastructure réseau professionnelle sur AWS — la fondation du projet final.*

Objectifs de la séance

À l'issue de cette séance, vous serez capable de :

1. **Créer** un VPC avec des subnets publics et privés via Terraform.
2. **Configurer** le routage réseau (Internet Gateway, NAT Gateway, Route Tables).
3. **Déployer** un bastion SSH comme unique point d'entrée.
4. **Accéder** à un serveur privé via le bastion (SSH ProxyJump).

Partie théorique

1. Le problème : tout est public

Jusqu'à S10, notre architecture ressemble à ça :

```
Internet ─── EC2 (IP publique)
                ↑
          SSH + HTTP ouverts au monde
```

C'est acceptable pour un TP, pas pour la production. Les bases de données, les services internes, les outils d'administration ne doivent pas être accessibles depuis internet.

2. L’architecture multi-tier

```
                          Internet
                              │
                              ┴
                     ┌──────────────────┐
                     │ Internet Gateway │
                     └──────────────────┘
                              ┬
                              │
              ┌───────────────┴────────────────┐
              │          VPC NovaSphere        │
              │          10.0.0.0/16           │
              │                                │
              │  ┌──────────────────────────┐  │
              │  │  Subnet Public           │  │
              │  │  10.0.1.0/24             │  │
              │  │                          │  │
              │  │  ┌──────────┐ ┌──────────┐ │  │
              │  │  │ Bastion │ │ Web (Nginx)│ │  │
              │  │  │ SSH only│ │ HTTP+SSH  │ │  │
              │  │  └──────────┘ └──────────┘ │  │
              │  └──────────────────────────┘  │
              │                                │
              │  ┌──────────────────────────┐  │
              │  │  Subnet Privé            │  │
              │  │  10.0.2.0/24             │  │
              │  │  (pas d'IP publique)      │  │
              │  │                          │  │
              │  │  ┌──────────┐            │  │
              │  │  │   BDD    │            │  │
              │  │  │ (futur)  │            │  │
```

```
│   │   └──────────┘                │   │
│   └──────────────────────────────┘   │
│                                      │
└──────────────────────────────────────┘
```

Composant	Rôle
VPC	Réseau virtuel isolé (10.0.0.0/16 = 65 536 IPs)
Subnet public	Serveurs accessibles depuis internet (IP publique)
Subnet privé	Serveurs internes (pas d'IP publique, accès via NAT)
Internet Gateway	Porte de sortie/entrée vers internet (subnet public)
NAT Gateway	Permet au subnet privé d'accéder à internet (mises à jour) sans être accessible depuis internet
Bastion	Unique point d'entrée SSH. On s'y connecte, puis on rebondit vers les serveurs privés.

3. Le SSH ProxyJump

Pour accéder à un serveur privé (pas d'IP publique), on passe par le bastion :

```
# Connexion directe au bastion
ssh -i ~/.ssh/novasphere ubuntu@<IP_BASTION>


# Connexion au serveur privé via le bastion (ProxyJump)
ssh -i ~/.ssh/novasphere -J ubuntu@<IP_BASTION> ubuntu@<IP_PRIVÉE>
```

L'option `-J` (ProxyJump) dit à SSH : « connecte-toi d'abord au bastion, puis depuis le bastion, connecte-toi au serveur privé ».

TP — Construire l'infrastructure réseau

Workflow Git :

```
cd ~/novasphere-infra
git switch -c feat/s11-vpc
```

Exercice 1 — Le VPC et les subnets (25 min)

Étape 1 — Créez un nouveau répertoire pour l'infra multi-tier :

```
cd ~/novasphere-infra/terraform
mkdir -p multi-tier
cd multi-tier
```

Étape 2 — Créez `providers.tf` :

```
# providers.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

Étape 3 — Créez `variables.tf` :

```
# variables.tf
variable "aws_region" {
  type    = string
  default = "eu-west-3"
}

variable "project_name" {
  type    = string
```

```
  default = "novasphere"
}

variable "vpc_cidr" {
  type    = string
  default = "10.0.0.0/16"
}

variable "public_subnet_cidr" {
  type    = string
  default = "10.0.1.0/24"
}

variable "private_subnet_cidr" {
  type    = string
  default = "10.0.2.0/24"
}

variable "ssh_public_key_path" {
  type    = string
  default = "~/.ssh/novasphere.pub"
}

variable "ssh_private_key_path" {
  type    = string
  default = "~/.ssh/novasphere"
}
```

Étape 4 — Créez `network.tf` :

```
# network.tf — VPC, Subnets, Routage

# --- VPC ---
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name    = "${var.project_name}-vpc"
    Project = "NovaSphere"
  }
}

# --- Internet Gateway ---
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name    = "${var.project_name}-igw"
    Project = "NovaSphere"
  }
}

# --- Subnet Public ---
resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidr
```

```
  map_public_ip_on_launch = ____________    # TODO 1 : true ou false ?
(les instances ont-elles une IP publique ?)
  availability_zone       = "${var.aws_region}a"


  tags = {
    Name    = "${var.project_name}-public"
    Project = "NovaSphere"
  }
}


# --- Subnet Privé ---
resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet_cidr
  availability_zone = "${var.aws_region}a"


  tags = {
    Name    = "${var.project_name}-private"
    Project = "NovaSphere"
  }
}


# --- Route Table Public ---
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id


  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
```

```
  }

  tags = {
    Name    = "${var.project_name}-public-rt"
    Project = "NovaSphere"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

# --- Elastic IP + NAT Gateway (pour le subnet privé) ---
resource "aws_eip" "nat" {
  domain = "vpc"

  tags = {
    Name    = "${var.project_name}-nat-eip"
    Project = "NovaSphere"
  }
}

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public.id

  tags = {
```

```
    Name    = "${var.project_name}-nat-gw"
    Project = "NovaSphere"
  }
}

# --- Route Table Privé ---
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }

  tags = {
    Name    = "${var.project_name}-private-rt"
    Project = "NovaSphere"
  }
}

resource "aws_route_table_association" "private" {
  subnet_id      = aws_subnet.private.id
  route_table_id = aws_route_table.private.id
}
```

Question : Pourquoi le subnet privé a-t-il une route vers le NAT Gateway plutôt que vers l'Internet Gateway ?

Réponse : _____

Exercice 2 — Security Groups + Instances (25 min)

Étape 1 — Créez `security.tf` :

```
# security.tf — Security Groups

# --- SG Bastion : SSH depuis internet ---
resource "aws_security_group" "bastion" {
  name        = "${var.project_name}-bastion-sg"
  description = "SSH uniquement depuis internet"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "SSH"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name    = "${var.project_name}-bastion-sg"
    Project = "NovaSphere"
```

```
  }
}

# --- SG Web : HTTP depuis internet + SSH depuis bastion ---
resource "aws_security_group" "web" {
  name        = "${var.project_name}-web-sg"
  description = "HTTP depuis internet, SSH depuis bastion"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description     = "SSH depuis bastion"
    from_port       = 22
    to_port         = 22
    protocol        = "tcp"
    security_groups = [___________]    # TODO 2 : Référence au SG du
bastion
  }

  egress {
    from_port   = 0
    to_port     = 0
```

```
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name    = "${var.project_name}-web-sg"
    Project = "NovaSphere"
  }
}
```

> **Note** : Le Security Group `web` n'accepte SSH que depuis le bastion (référence au SG, pas à une IP). C'est plus sûr et plus flexible qu'un CIDR.

Étape 2 — Créez `instances.tf` :

```
# instances.tf — Instances EC2

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

```
resource "aws_key_pair" "novasphere" {
  key_name   = "${var.project_name}-key"
  public_key = file(var.ssh_public_key_path)
}

# --- Bastion ---
resource "aws_instance" "bastion" {
  ami                     = data.aws_ami.ubuntu.id
  instance_type           = "t3.micro"
  key_name                = aws_key_pair.novasphere.key_name
  subnet_id               = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.bastion.id]

  tags = {
    Name    = "${var.project_name}-bastion"
    Project = "NovaSphere"
    Role    = "bastion"
  }
}

# --- Serveur Web ---
resource "aws_instance" "web" {
  ami                     = data.aws_ami.ubuntu.id
  instance_type           = "t3.micro"
  key_name                = aws_key_pair.novasphere.key_name
  subnet_id               = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
```

```
  tags = {
    Name    = "${var.project_name}-web"
    Project = "NovaSphere"
    Role    = "web"
  }
}
```

Étape 3 — Créez `outputs.tf` :

```
# outputs.tf
output "bastion_public_ip" {
  value = aws_instance.bastion.public_ip
}

output "web_public_ip" {
  value = aws_instance.web.public_ip
}

output "vpc_id" {
  value = aws_vpc.main.id
}
```

Étape 4 — Déployez :

```
terraform init
terraform plan
terraform apply -auto-approve
```

> **Question** : Combien de ressources Terraform crée-t-il ? Listez les types.
>
> Réponse : _____

Exercice 3 — Test du bastion et ProxyJump (15 min)

Étape 1 — Connectez-vous au bastion :

```
ssh -i ~/.ssh/novasphere ubuntu@$(terraform output -raw
bastion_public_ip)
```

Étape 2 — Depuis le bastion, connectez-vous au serveur web (via IP privée) :

```
# Sur le bastion :

# Le serveur web a une IP publique dans ce TP, mais en production

# il serait dans le subnet privé. On simule le rebond.

exit
```

Étape 3 — Utilisez ProxyJump depuis votre poste :

```
BASTION_IP=$(terraform output -raw bastion_public_ip)

WEB_IP=$(terraform output -raw web_public_ip)



ssh -i ~/.ssh/novasphere -J ubuntu@${BASTION_IP} ubuntu@${WEB_IP}
```

Étape 4 — Configurez SSH pour simplifier :

Ajoutez dans `~/.ssh/config` :

```
Host bastion-novasphere

    HostName <IP_BASTION>

    User ubuntu

    IdentityFile ~/.ssh/novasphere


Host web-novasphere

    HostName <IP_WEB>

    User ubuntu

    IdentityFile ~/.ssh/novasphere

    ProxyJump bastion-novasphere

# Maintenant, une seule commande :

ssh web-novasphere
```

Question : Pourquoi le ProxyJump est-il plus sûr qu'ouvrir SSH directement sur le serveur web depuis internet ?

Réponse : _____

Exercice 4 — Nettoyage et commit (5 min)

```
cd ~/novasphere-infra/terraform/multi-tier

terraform destroy -auto-approve


cd ~/novasphere-infra

git add .

git commit -m "feat: infrastructure multi-tier VPC, bastion, web"


git switch main

git merge feat/s11-vpc

git branch -d feat/s11-vpc

git tag -a v0.11.0 -m "S11: VPC multi-tier + bastion"

git push && git push --tags
```

Important : Le NAT Gateway coûte ~$0.05/heure. Vérifiez que `terraform destroy` a bien tout supprimé, y compris la NAT Gateway et l'Elastic IP.

Critères de réussite

#	Critère	Vérifié ?
1	Le VPC `novasphere-vpc` est créé avec 2 subnets	
2	Le bastion est accessible en SSH depuis internet	
3	Le serveur web accepte HTTP et SSH depuis le bastion uniquement	
4	ProxyJump fonctionne (connexion au web via le bastion)	
5	`terraform destroy` supprime TOUTES les	

#	Critère	Vérifié ?
	ressources (dont NAT + EIP)	
6	Le code est commité avec le tag `v0.11.0`	

Projet Final (Séances 12 à 14)

Contexte

NovaSphere passe en production. Le CTO lance un sprint final :

« Vous avez 3 séances pour livrer une infrastructure complète, automatisée, sécurisée et documentée. Je veux pouvoir détruire toute l'infra et la recréer en une seule commande. Zéro intervention manuelle. »

Architecture minimale requise

```
                        Internet
                            │
                            ┴
                  ┌─────────────────┐
                  │ Internet Gateway │
                  └─────────────────┘
                            ┬
                            │
              ┌─────────────┴─────────────┐
              │          VPC NovaSphere   │
              │                           │
              │  ┌─────────────────────┐  │
              │  │  Subnet Public      │  │
              │  │  ┌─────────┐ ┌──────┐ │  │
              │  │  │ Bastion │ │ Web  │ │  │
              │  │  └─────────┘ └──────┘ │  │
              │  └─────────────────────┘  │
              │                           │
              │  ┌─────────────────────┐  │
```

```
              │  │  Subnet Privé            │  │
              │  │  ┌──────────┐            │  │
              │  │  │ Service  │            │  │
              │  │  │ interne  │            │  │
              │  │  └──────────┘            │  │
              │  └──────────────────────────┘  │
              └────────────────────────────────┘
```

Exigences du projet

- VPC custom avec subnet public + privé, routage complet
- Bastion SSH comme unique point d'entrée
- Serveur web Nginx accessible en HTTP, SSH uniquement via bastion
- Security Groups stricts
- Rôle Ansible pour la configuration du serveur web
- Variables externalisées (Terraform + Ansible)
- Secrets chiffrés avec Ansible Vault
- Script `deploy.sh` : pipeline complet sans intervention manuelle
- Script `destroy.sh` : destruction propre de toute l'infra
- Documentation `README.md` complète
- Historique Git propre avec branches et messages conventionnels

Structure attendue du dépôt

```
novasphere-infra/
├── README.md
├── deploy.sh
├── destroy.sh
├── terraform/
│   └── multi-tier/
│       ├── providers.tf
│       ├── variables.tf
│       ├── network.tf
│       ├── security.tf
│       ├── instances.tf
│       ├── outputs.tf
```

```
│       └── templates/
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/
│   ├── vault/
│   │   └── secrets.yml
│   ├── playbook.yml
│   └── roles/
│       └── webserver/
└── packer/  (bonus)
```

Déroulement

Séance	Activité
S12	Formation binômes, conception architecture, début développement
S13	Développement encadré, intégration TF + Ansible, tests pipeline
S14	Derniers ajustements + soutenance (10 min par binôme : archi + démo live + questions)